claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-a1011a7004b43f662.jsonl`
- SHA-256: `65ec259f27c575a1ca0d67e7fa4d19fb227df15185395912c88bc4a13f5ade4a`
- Source modified: `2026-06-07T09:20:30+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T09:17:12.494Z)

Adversarially verify this regenerated CODE_MAP against the REAL repo at C:/Users/avidu/Projects/badminton-highlight-indexer. For EVERY @path anchor, confirm the file exists and is git-tracked at that exact path (use git ls-files / Read). For every ::symbol, confirm it exists in that file. Flag any anchor that does NOT resolve, any reference to an OLD path (backend/validators, backend/indexer, backend/database, backend/stitcher, backend/detectors, backend/audio), and any clearly wrong claim. Return ok=true only if nothing is broken.

=== CODE_MAP ===
Now I have the full picture of the existing structure. Let me regenerate the complete CODE_MAP.md, applying the path migrations, new subsystems (config, reporting), refreshed test map, and corrected gotchas/ramp paths.

CODE_MAP.md

Purpose: cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

Regenerate: rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

LEGEND
- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `§` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

1. PIPELINE

1A. Runtime: video file -> highlight reel

```
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-defaults, never fatal)
                                          @backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig
  -> video_id = MD5(whole file)           @backend/main.py::get_file_md5  (? chunk size varies
```

per entrypoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE file so values agree)
  -> validate @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
       FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/pipeline/validators/__init__.py)
       [validation FAIL -> persist status="failed", return early; report STILL emitted in finally]
  -> db.add_video(video_id, filepath, status, [r.dict()])   @backend/infrastructure/database.py
  -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
  -> seg = segmenter_registry.get(seg_name)(db, global_config)   ? @backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
  -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
       [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]
  -> finally: emit_indexer_report(...)   @backend/reporting/__init__.py (? ALWAYS runs ? even on validation failure / exception; NEVER raises; grafts response["reports"])
  -> select segment ids -> [(start,end)] (+ stitching padding)
  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name) @backend/pipeline/stitcher/compiler.py
       per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4
```

Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):

```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
 -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted via db.add_candidate_segment]
 -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip, prompt)   ? provider_registry
 -> P4 db.add_play_segment + db.link_candidate_to_segment
```

Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) -> add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

1B. WASB shuttle substrate (P2 of wasb_hybrid)

```
WasbHybridSegmenter.process_video                @backend/pipeline/segmenters/wasb_hybrid.py
 -> WasbRunner.healthcheck()  (gate; not ok -> return [])  @backend/pipeline/detectors/wasb_runner.py
 -> WasbRunner.run_predict(video, out_dir)
      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python wasb_infer.py ...'`
      -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
 -> TrackNetRunner.parse_trajectory_csv(csv)      @backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
 -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING BRAIN]
```

(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback (30.0, 1280.0).)

1C. Calibration / eval flow (NO pipeline run unless told)

```
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage ? {candidates, final}
  -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
  -> harness.evaluate -> EvalReport -> report_to_dict
       -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
       -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py  [CI
```
```

gate]
Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate}
@backend/eval/calibration.py
Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier)
@backend/eval/{calibrate_local,distill_local}.py
Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
Gemini refinement driver (tuning, standalone): gemini_refine.{refine_window,full_video_rallies}
@backend/eval/gemini_refine.py
Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
```
Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).

---

## 2. SUBSYSTEMS

### 2.0 Orchestration & config

- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`.
  - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
  - `::process_video` `@POST /api/process` -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read).
  - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
  - ? `main()` runtime-override hack: `global_config._runtime_overrides = {"output_dir": args.output_dir}` ? output dir lives ONLY on this dynamic attr (works because `AppConfig` is `extra="allow"`), NOT in any config model; `db_path = os.path.join(args.output_dir, output.db_name)`.
  - ? `compile_highlights` reads `getattr(global_config.output,"output_dir","output")` but `OutputConfig` has **no `output_dir`** -> ALWAYS falls back to `"output"` (the `_runtime_overrides` value is NOT consulted here).
  - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), `default_segmenter` fallback `'yolo_hybrid'`, raw `json.load` config (bypasses `backend.config`), `use_cache` defaults `True` (opposite of model/config `False`), skips validation.
- `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` reads `config.json` via raw `json.load` (bypasses `backend.config`).
- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label); `::load_config` is its OWN raw `json.load` dict loader; `segmenter_cls(db, cfg)` is called with a **raw dict** cfg (not AppConfig).
- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::_default_db_path` again raw `json.load`.

- `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config
```

import load_config, AppConfig`.
- `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
  - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance); also what lets `main.py` graft `_runtime_overrides`.
  - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
  - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
  - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_fps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`); `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`). `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`). `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`; ? NO `output_dir` field ? runtime-only). `::StitchingConfig` (`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal of 5 sports. `::Config = AppConfig` alias.
- `@backend/config/loader.py` ? load/save with defaults-merge.
  - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
  - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole defaults dict for that section (Pydantic then re-applies field defaults for missing leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).
  - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
- `@config.json` ? on-disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.
- `@setup.py` ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA).

2.1 Segmenters (under pipeline/segmenters) ?

- `@backend/pipeline/segmenters/base.py`
  - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
  - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).
- `@backend/pipeline/segmenters/__init__.py` ? imports all 5 (`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.
- `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config

`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs `{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit (copy-paste drift).
- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig` from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`; `detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
- `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections). `::_lazy_load_model` (lazy `torchvision` import so module/tests load without it), `::_extract_action_windows_from_chunk`, `::_make_tracker` (per-chunk ByteTrack ? track IDs chunk-local), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU); `yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated aliases still honored.
- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning table).
- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.

2.2 Detectors & windowing (WSL-quarantined)

- `@backend/pipeline/detectors/tracknet_runner.py`
  - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
  - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled), `::TrajectoryPoint`(frame,visible,x,y).
  - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names `<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever). `::_stage_video` "OK" sentinel.
- `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `_wsl_bash` duplicated (drift risk).
- `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights` compared raw), `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array), `::_build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW), `::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `--fresh` ignores cache.
- `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure, no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'), `::count_net_crossings` (sign changes vs median net; deadband jitter zone), `::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds). `::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is `(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /

distill_local / export_wasb_segments.

2.3 Eval & calibration (all pure core; I/O in CLI drivers)

- `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`, `::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN) for mean_iou/boundary errors.
- `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS), `::write_scaffold`.
- `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. § `PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap >~0.1` = overfit alarm.
- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`, `::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over `trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts, make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local, export_wasb_segments, gemini_refine, audio/e1_probe).
- `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing + net-crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn` (adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a labels file yields no rallies), `::run`/`::main`. § `LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`; manifest JSON shared with distill_local.
- `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison + optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`,`::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice` (lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
- `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
- `@backend/eval/harness.py` ? DB-pred scorer (no re-run). `::STAGES=('candidates','final')`, `::get_predictions` (candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown stage raises ValueError; reused by regression.py), `::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for batch.aggregate), `::format_report_text`.
- `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES` (5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default 0.0), `::label_candidates` (same IoU `match_intervals` as evaluator), `::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
- `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn). `fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default; stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid` clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
- `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT: duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal), `::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`), `::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-one-video-out needs ?2 videos; final model trained on all.
- `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT padded clips (precision lever; eval/tuning, NOT the live pipeline). `::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (? FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`, `::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises SystemExit); imports `BASELINE` from calibrate_wasb + `TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.
- `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`, `::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`, `::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;

"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
`backend.annotations` NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).

2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)

- `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure numpy +
stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k·std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted
(PR #35).
- `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.

2.4 Stitcher / tools / utils

- `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.
- `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-
tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp`.
- `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure = "unknown"),
`::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at nearest
keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
- `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s; ? raw-
px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention inflates
IoU), `::get_center/get_distance/get_velocity`.

2.5 Validators ?

- `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult` (pydantic:
validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ? registration
order = exec order; ? no dedup), `::registry` (singleton). § `validate(video_path, config,
context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps each
validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
- `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
- `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ? **PRODUCER** of
`context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring
`'mp4'` (so MOV-family passes); external `ffprobe`.
- `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
- `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES
(too short).
- `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator` (`scene_cut_check`) ?
OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps at 100 SAMPLED frames
(~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail threshold.
- `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold`(default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?

magic absolute, not resolution-normalized.
- `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ? HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`; 3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport -> empty cfg, falls back to defaults, does NOT fail.

2.6 Sports / Providers / Annotations ?

- `@backend/sports/base.py::SportProfile` ABC (`name`,`get_prompt`,`get_relevance_config`); `@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg); `@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ? `register` instantiates profile to read `.name`). ? **add a sport**: subclass `SportProfile`, `sport_registry.register(MyProfile)`. § prompt emits JSON array `{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS; `get()` returns a fresh instance each call.
- `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`); `@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`; upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry` (+`::ProviderRegistry`, auto-registers `'gemini'`). ? **add a provider**: subclass, `provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key, model_name)`. § `analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list, metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)` on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible if process dies mid-run.
- `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`), `::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`, `::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off `provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` -> LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`), `annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. § `fetch -> List[Interval]` sorted; contract says RAISE (not `[]`) on unavailable.
- `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the resolved CSV path; `_resolve` -> `<csv_dir>/<video_ref>.csv`; -> `backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
- `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; GS-4 stub; injectable `::Labeler`, in-process `_jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only warns, does not block; default `_not_configured` RAISES NotImplementedError). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as `"auto"` only because no-arg ctor succeeds.
- `@backend/annotations/__init__.py` ? ? `noqa F401` imports of file_provider/automated_provider are load-bearing (removing de-registers providers).

2.7 Infrastructure (DB) ?

- `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables (`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in `::_init_db` (currently =1, `version<1` creates `candidate_segments`). `::_get_connection` (? re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped). Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_segment`,`link_candidate_to_segment`,`query_candidate_segments(detector=,only_unlabeled=)`,`query_play_segments`,`get_video`, `clear_video_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point (common fields columns, detector-specifics in `stats`/`detection_params` JSON; index `idx_candidates_video_detector`). ? `add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest; events column is `type` not `event_type`; `query_play_segments` filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow exceptions.

2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)

- `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path, config, val_results, val_context=None, segmenter_requested=, segmenter_actual=, was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed config (`config.model_dump(...)` if `hasattr model_dump`); short-circuits to None if unavailable/disabled; pulls play_segments/candidates from `db` (read-only `query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to `harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s

`finally:`); writes `<video>.report.json`/`.report.md`/`.summary.md` next to video.
`::OBSREPORT_AVAILABLE` ? soft-dep gate (True only if `import obsreport` AND `from . import harvest` both succeed). `::report_enabled(config)` reads `config["report"]["enabled"]` default True.
- `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the footgun RULES live in spec.yaml, not here). `::record_run(...)` ? top-level assembler (does NOT write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`, runs 4 stage builders + `_highlights`, `rec.finalize()`.
`::_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}` ? (add new AI segmenters here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; § `fps_source ? {unknown,probed,fallback}`), `::_validation_stage`, `::_segmentation_stage` (§ `details.{segmenter_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_requested,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"), `::_ai_stage` (§ `details.{ai_based,provider,model,api_key_present}`; metric `confirmed_segments`), `::_highlights` (coverage_pct). ? `record_run` fps fallback: `fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners). `::_SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module.
- `@backend/reporting/spec.yaml` ? § declarative report layout + footgun engine consumed by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections` (ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each `{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd; `path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule. Rules: `silent_provider_noop`(error: no api key + 0 segs), `ai_empty_vs_no_rallies`(warn), `fps_fallback_used`(warn), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info), `reingest_replaced_prior`(info), `validation_failed`(error: halts indexing).

---

3. DATA CONTRACTS (canonical shapes ? one place)

§ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`): `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords. -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).

§ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo `_extract_action_windows_from_chunk`):
`{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float, mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.

§ **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
`{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
`detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_threshold,min_action_window_duration}`.

§ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
`{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server, receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count, shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare `model_name`; motion adds random `intensity/avg_speed_kmh`.

§ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time, end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start; `_candidate_id` stamped internally).

§ **GT / golden CSVs** (TWO conventions ? do not conflate):
- generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal or MM:SS/HH:MM:SS; RAISES on bad row.

- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`, `export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`; SILENTLY skips bad rows.

§ **det_raw.jsonl** (one window/line): `{"w":<window_index>, "f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`); first violation/JSONDecodeError -> truncate-heal.
§ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in, frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
§ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv}, ...]}`; ? DB key = file MD5, manifest `video_id` is a label.

§ **eval primitives**: `Interval=Tuple[float,float]` (start<end). `Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP), unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.

§ **WSL detector candidate** (in-WSL, model<->tracker): `{'xy':np.array([x,y]), 'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST rebuild `xy` as np.array (tracker does `np.linalg.norm`).

§ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}` ? onset-envelope peak.

§ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages `validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail}; flags `{code,faq_ref}`; `video_meta.fps_source ? {probed,fallback}`. ? stage/metric/detail names are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync: `ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmentation.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinget}`, `validation.status`, `video_meta.fps_source`.

---

4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims below are ur

- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval', overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from cfg["model"] at runtime in wasb_infer.run.)
- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results, hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()` resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls `build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y` ? consistent.)
- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path, center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in wasb_infer.run.)
- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.

---

5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified here)

- Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact file used.
- `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer

`start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` + `@run_phase3_eval.py`.
- Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) -> consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` / `@backend/tools/rally_slicer.py`.

---

6. GOTCHAS (cross-cutting footguns)

- **MD5 keying:** `video_id` = MD5 of WHOLE file everywhere ? but each entrypoint uses a different chunk SIZE (64KB/1MB/8KB); values agree, code does not. Manifest `video_id` is a label, NOT the DB key.
- **Two config-access idioms coexist:** `backend/main.py` uses typed `AppConfig` (attribute access + `getattr` fallbacks); all four `run_*.py` scripts read `config.json` via raw `json.load` into a dict and bypass `backend.config`. Default-drift between model / config.json / per-script literal fallbacks is the recurring footgun.
- **Segmenter default defined in 5 places, 3 values:** model `yolo_hybrid`, config.json `gemini`, main.py fallback `motion`, phase3 default `motion`, process_and_stitch fallback `yolo_hybrid`. You may run a different detector than expected.
- **`AppConfig.get` must return dict sections, not models:** the legacy `.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also silently keeps typo'd config keys (no rejection).
- **`output_dir` is runtime-only:** lives on `global_config._runtime_overrides` (set in `main()`), NOT in any config model. `OutputConfig` has no `output_dir` field, so `compile_highlights` always falls back to literal `"output"`.
- **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also returns `[]` on real failure -> empty != "no rallies". The reporting `silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in the report.
- **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ? absent -> no-op.
- **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops firing. Add rules in spec.yaml, not code.
- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
- **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo modules).
- **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
- **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in `output/<db_name>` before any eval/regression.
- **Validator ordering coupling:** exec order is Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless `format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6 correlation are NOT config-overridable.
- **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue `PRAGMA foreign_keys=ON` per connection.
- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs tracknet's `is None` check.
- **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs `backend.annotations` (provider registry). regression.py imports the latter.
- **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`;

`harness.get_predictions` reused by `regression.regress`.
- **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
- **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
- **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
- **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` + `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2× -> not adopted for fusion.

---

7. RAMP PATHS (to do X -> open Y::symbol)

Task	Open
Change a config default / add a config field	`@backend/config/models.py` (single source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override (default_segmenter)
Read config in new code	`from backend.config import load_config, AppConfig` (`@backend/config/__init__.py`); attribute access, NOT raw `json.load`
Tune candidate windowing (velocity/merge/min-dur)	`@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration`
Add a segmenter	subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`; `segmenter_registry.register(...)` at module bottom; import in `@backend/pipeline/segmenters/__init__.py`; if AI-based add to `@backend/reporting/harvest.py::_AI_SEGMENTERS`
Change stitching padding	`config.json stitching.{begin_padding,end_padding}`; impl `@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror `@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING`
Add a sport	subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file
Add an AI provider	subclass `@backend/providers/base.py::AIVideoProvider`; `provider_registry.register('name', Cls)`
Add an annotation/GT tier	subclass `@backend/annotations/base.py::AnnotationProvider`; register + import in `@backend/annotations/__init__.py`
Add / change a report footgun rule	`@backend/reporting/spec.yaml::rules` (declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in `@backend/reporting/harvest.py`
Tweak what the report records	`@backend/reporting/harvest.py::record_run` + stage builders; emit point is `@backend/main.py` `finally:` -> `@backend/reporting/__init__.py::emit_indexer_report`
Run calibration (WASB thresholds)	`@backend/eval/calibrate_wasb.py::main` (`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit`
Distill Gemini -> local (thresholds / learned)	`@backend/eval/calibrate_local.py::main` (thresholds) OR `@backend/eval/distill_local.py::main` (learned classifier); both `--manifest`
Refine WASB candidates with Gemini (tuning)	`@backend/eval/gemini_refine.py::main` (`--full-video` skips `--trajectory`; needs `GEMINI_API_KEY`)
Export tuned rally segments / clips	`@backend/eval/export_wasb_segments.py::main` (`--slice` needs `--video`)
Run a video end-to-end	`@backend/main.py::main` (`--video-path` or `--server`) or `@run_process_and_stitch.py::main` (edit hardcoded paths)
Score DB preds vs GT (one video)	`@run_eval.py::main`
Batch score a collector manifest	`@run_phase3_eval.py::main` (`--manifest`)
Guard a regression (CI)	`@run_regression.py::main` (`--video-id`); exit 1=regress, 2=missing DB; snapshot via `--update-baseline`
Run the whole test suite	`@run_all_tests.py` (repo root); or `pytest tests/` directly
Resume a crashed WASB pass	nothing to do ? `@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from `det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart
Add a validator	subclass `@backend/pipeline/validators/base.py::VideoValidator`; `registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order; producers before consumers)
Add a DB column/table	`@backend/infrastructure/database.py::Database._init_db` migration ladder + bump `PRAGMA user_version` (else version-assert tests fail)

| Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
| Probe an audio signal (diagnostic) | `@backend/eval/audio/e1_probe.py::main`; detector in
`@backend/pipeline/audio/hit_detector.py` |

---

8. TEST MAP (`tests/test_*.py` -> subsystem)

All pure-logic / mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked.
`@run_all_tests.py` (repo root) is the suite runner.
Test	Covers
`test_base.py`	`pipeline/segmenters/base.SegmenterRegistry` (register/get/list via
DummySegmenter)
`test_yolo_hybrid.py`	`pipeline/segmenters/yolo_hybrid.HybridYoloSegmenter`
(torchvision+ByteTrack; `_PERSON_CLASS_ID` filter; candidate link; `store_yolo_candidates=False`
skip)
`test_tracknet_hybrid.py`	`pipeline/segmenters/tracknet_hybrid.TrackNetHybridSegmenter`
(4-phase; healthcheck-abort; `_probe_fps_width` fallback)
`test_gemini.py`	`pipeline/segmenters/gemini.GeminiPlaySegmenter` (single vs chunked path,
per-chunk offset, `shot_count`<-`shuttle_exchange_count`)
`test_tracknet_runner.py`	`pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`,
`from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck,
run_predict, stage)
`test_rally_gate.py`	`pipeline/detectors/rally_gate` (axis/net estimate, crossing count,
gate kills single-toss)
`test_wasb_cache.py`	`pipeline/detectors/wasb_infer` resumable cache (serialize, contiguity
heal, manifest invalidation, atomic writes, `default_cache_dir`)
`test_eval_metrics.py`	`eval.metrics` IoU/matching/score/pr_curve
`test_eval_annotations.py`	`eval.annotations` GT CSV parse/validate, `write_scaffold`
`test_eval_harness.py`	`eval.harness` DB-pred fetch + evaluate + report_to_dict; also
`@run_eval.py::main`/`get_file_md5` (exit codes, `--scaffold`/`--json`)
`test_eval_batch.py`	`eval.batch` path resolve / stage select / aggregate (micro/macro,
zero-div)
`test_calibration.py`	`eval.calibration`
pct_improvement/kfold/select_best/cross_validate/LOVO/improvement_report
`test_calibrate_wasb.py`	`eval.calibrate_wasb` load_golden_gts/make_predict_fn/grid/run,
BASELINE
`test_calibrate_local.py`	`eval.calibrate_local` local grid + crossing gate, leave-one-
video-out
`test_export_wasb_segments.py`	`eval.export_wasb_segments` segment/comparison CSV schemas,
mmss; export->slicer reuse
`test_eval_training.py`	`eval.training` label_candidates / feature extraction /
build_training_set
`test_eval_classifier.py`	`eval.classifier.LogisticRegression`
fit/predict_proba/balanced/JSON round-trip
`test_eval_regression.py`	`eval.regression` baseline store/compare/gate; also
`@run_regression.py::main` (exit-code CI gate)
`test_distill_local.py`	`eval.distill_local` build_candidates / predict_rallies / learned-
vs-baseline
`test_gemini_refine.py`	`eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API)
`test_hit_detector.py`	`pipeline/audio/hit_detector` (detect_hits, onset_envelope,
_moving_stats) + `eval/audio/e1_probe.cluster_hits`
`test_rally_slicer.py`	`tools.rally_slicer.compute_clip_windows` (pure) + label parse
`test_video.py`	`utils.video` duration/chunk (subprocess mocked)
`test_geometry.py`	`utils.geometry` center/distance/velocity/normalised-velocity/IoU
`test_database.py`	`infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard
`test_config.py`	`config` (load_config defaults, save_default_config, AppConfig defaults
pinned, `extra` tolerance, `.get()` dict-compat)
`test_reporting_harvest.py`	`reporting` (`emit_indexer_report`, `harvest.record_run`); ?
`importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-AppConfig +
`report.enabled=False`

`test_reporting_spec_faq.py`	`reporting/spec.yaml` <-> `README.md` cross-ref integrity (every `faq_ref` resolves; actionable rules cite a faq_ref); ? `importorskip("obsreport")`
`test_validators_base.py`	`pipeline/validators/base.ValidationRegistry` (register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
`test_sport_profiles.py`	`sports` registry + BadmintonSportProfile (prompt contains "decimal seconds"/chunk value; relevance config keys)
`test_providers.py` / `test_provider_gemini.py`	`providers` registry + GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete)
`test_annotation_providers.py` / `test_automated_provider.py`	`annotations` registry, FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract -> `eval.metrics.score`
(no dedicated test)	individual `pipeline/validators/*` impls, `pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`, `pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`, `eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly
? stale `.pyc` only	`tests/__pycache__/test_gemini_api.*` ? no live source exists

2. assistant (2026-06-07T09:17:15.808Z)

I'll verify this CODE_MAP against the real repo. Let me start by getting the full list of git-tracked files and checking the directory structure.

3. user (2026-06-07T09:17:16.844Z)

```
.coveragerc
.gitignore
README.md
TEST_RUN_SUMMARY.md
backend/annotations/__init__.py
backend/annotations/automated_provider.py
backend/annotations/base.py
backend/annotations/file_provider.py
backend/api/static/app.js
backend/api/static/index.html
backend/api/static/style.css
backend/config/__init__.py
backend/config/loader.py
backend/config/models.py
backend/eval/__init__.py
backend/eval/annotations.py
backend/eval/audio/__init__.py
backend/eval/audio/e1_probe.py
backend/eval/batch.py
backend/eval/calibrate_local.py
backend/eval/calibrate_wasb.py
backend/eval/calibration.py
backend/eval/classifier.py
backend/eval/distill_local.py
backend/eval/export_wasb_segments.py
backend/eval/gemini_refine.py
backend/eval/harness.py
backend/eval/metrics.py
backend/eval/regression.py
backend/eval/training.py
backend/infrastructure/database.py
backend/main.py
backend/pipeline/audio/__init__.py
backend/pipeline/audio/hit_detector.py
backend/pipeline/detectors/__init__.py
backend/pipeline/detectors/rally_gate.py
backend/pipeline/detectors/tracknet_runner.py
backend/pipeline/detectors/wasb_infer.py
backend/pipeline/detectors/wasb_runner.py
backend/pipeline/segmenters/__init__.py
```

backend/pipeline/segmenters/base.py
backend/pipeline/segmenters/gemini.py
backend/pipeline/segmenters/motion.py
backend/pipeline/segmenters/tracknet_hybrid.py
backend/pipeline/segmenters/wasb_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
backend/pipeline/stitcher/compiler.py
backend/pipeline/validators/__init__.py
backend/pipeline/validators/base.py
backend/pipeline/validators/blur.py
backend/pipeline/validators/format.py
backend/pipeline/validators/pts_continuity.py
backend/pipeline/validators/relevance.py
backend/pipeline/validators/resolution_fps.py
backend/pipeline/validators/scene_cut.py
backend/providers/__init__.py
backend/providers/base.py
backend/providers/gemini.py
backend/reporting/__init__.py
backend/reporting/harvest.py
backend/reporting/spec.yaml
backend/sports/__init__.py
backend/sports/badminton.py
backend/sports/base.py
backend/tools/__init__.py
backend/tools/rally_slicer.py
backend/utils/__init__.py
backend/utils/geometry.py
backend/utils/video.py
config.json
docs/AUDIO_E1_FINDINGS.md
docs/AUDIO_RALLY_DETECTION_PLAN.md
docs/BACKLOG.md
docs/CODE_MAP.md
docs/COMMERCIAL_READINESS_REVIEW.md
docs/DECISIONS.md
docs/EVALUATION.md
docs/GOLDEN_SET_IMPLEMENTATION.md
docs/GOLDEN_SET_PHASE1_VIDEOS.md
docs/GOLDEN_SET_VENDORING.md
docs/HOW_RALLY_DETECTION_WORKS.md
docs/LOW_REGRET_MOVES_PLAN.md
docs/MONETIZATION_AUDIT.md
docs/RALLY_DETECTION_RESEARCH.md
docs/ROADMAP.md
docs/TRACKNET_WSL_SETUP.md
docs/VIDEO_RECORDING_GUIDELINES.md
pytest.ini
requirements.txt
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
run_regression.py
scratch/analyze_motion.py
scratch/clean_state.py
scratch/diagnose_rally.py
scratch/gemini_video_debugger.py
scratch/list_models.py
scratch/test_ai_segmenter.py
scratch/test_thresholds.py
scripts/generate_khelacharya_flyer.py
setup.py
tests/__init__.py
tests/test_annotation_providers.py
tests/test_automated_provider.py

```
tests/test_base.py
tests/test_calibrate_local.py
tests/test_calibrate_wasb.py
tests/test_calibration.py
tests/test_config.py
tests/test_database.py
tests/test_distill_local.py
tests/test_eval_annotations.py
tests/test_eval_batch.py
tests/test_eval_classifier.py
tests/test_eval_harness.py
tests/test_eval_metrics.py
tests/test_eval_regression.py
tests/test_eval_training.py
tests/test_export_wasb_segments.py
tests/test_gemini.py
tests/test_gemini_refine.py
tests/test_geometry.py
tests/test_hit_detector.py
tests/test_provider_gemini.py
tests/test_providers.py
tests/test_rally_gate.py
tests/test_rally_slicer.py
tests/test_reporting_harvest.py
tests/test_reporting_spec_faq.py
tests/test_sport_profiles.py
tests/test_tracknet_hybrid.py
tests/test_tracknet_runner.py
tests/test_validators_base.py
tests/test_video.py
tests/test_wasb_cache.py
tests/test_yolo_hybrid.py
```

4. assistant (2026-06-07T09:17:20.285Z)

The directory structure already uses the new `pipeline/`, `infrastructure/`, `config/`, `reporting/` layout. Now let me verify the symbols. Let me read the key files in parallel, starting with the config subsystem and core pipeline files.

5. user (2026-06-07T09:17:21.234Z)

```
1       """Configuration package.
2
3       Public API:
4           from backend.config import load_config, AppConfig, Config
5
6       This package is the canonical place for all configuration access.
7       """
8
9       from .loader import load_config, save_default_config
10      from .models import AppConfig, Config, IndexingConfig, ValidationConfig  # re-export
common ones
11
12      __all__ = [
13          "load_config",
14          "save_default_config",
15          "AppConfig",
16          "Config",
17          "IndexingConfig",
18          "ValidationConfig",
19      ]
20
```

6. user (2026-06-07T09:17:21.709Z)

```
"""Pydantic models for the application configuration.

This module defines the canonical, validated, typed configuration for the
Badminton Highlight Indexer. Built-in defaults live here (single source of truth).

The models support graceful migration from old config.json files via extra="allow".
"""

from __future__ import annotations

from typing import Any, Literal

from pydantic import BaseModel, Field, field_validator

Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]


class ValidationRelevanceConfig(BaseModel):
    court_green_lower: list[int] = Field(default=[35, 40, 40])
    court_green_upper: list[int] = Field(default=[85, 255, 255])
    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


class ValidationConfig(BaseModel):
    min_resolution_width: int = 1280
    min_resolution_height: int = 720
    min_fps: float = 29.9
    min_blur_threshold: float = 20.0
    max_scene_cuts_per_minute: float = 0.5
    relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)


class TrackNetConfig(BaseModel):
    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/TrackNetV4"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "TrackNetV4"
    weights_path: str = ""
    queue_length: int = 5
    stage_in_wsl: bool = True
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 0
    velocity_threshold: float = 0.02


class IndexingConfig(BaseModel):
    default_segmenter: str = "yolo_hybrid"
    use_gpu: bool = True
    motion_threshold: float = 0.08
    min_segment_duration: float = 3.0
    dead_time_merge_gap: float = 2.0
    chunk_duration: float = 90.0
    chunk_overlap: float = 10.0
    detector_model: str = "fasterrcnn_resnet50_fpn"
    detector_score_threshold: float = 0.5
    yolo_velocity_threshold: float = 0.15
    yolo_frame_skip: int = 3
    yolo_tracker: str = "bytetrack.yaml"
    yolo_parallel_chunks: bool = False
    yolo_chunk_workers: int = 2
    min_action_window_duration: float = 2.0
    log_yolo_candidates: bool = True
    store_yolo_candidates: bool = True
    ai_handoff_padding: float = 2.0
```

```
65          ai_max_workers: int = 5
66
67          tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69          @field_validator("default_segmenter")
70          @classmethod
71          def segmenter_must_be_registered(cls, v: str) -> str:
72              # Registry check happens at runtime in main/segmenter selection.
73              # Here we just allow any string for forward compatibility.
74              return v
75
76
77      class OutputConfig(BaseModel):
78          default_highlights_name: str = "highlights.mp4"
79          db_name: str = "sports_indexer.db"
80
81
82      class StitchingConfig(BaseModel):
83          begin_padding: float = 0.5
84          end_padding: float = 0.5
85          use_cache: bool = False
86          min_shot_count: int = 1
87
88
89      class AppConfig(BaseModel):
90          """Root configuration object.
91
92          All runtime configuration should be accessed via an instance of this class
93          (or the loader) rather than raw dict lookups.
94          """
95
96          sport: Sport = "badminton"
97          ai_provider: str = "gemini"
98          ai_model: str = "gemini-2.5-flash"
99
100         validation: ValidationConfig = Field(default_factory=ValidationConfig)
101         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
102         output: OutputConfig = Field(default_factory=OutputConfig)
103         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
104
105         # Allow extra keys during transition so existing config.json files do not break.
106         model_config = {
107             "extra": "allow",
108             "validate_assignment": True,
109         }
110
111         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
112         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
113         # attribute access / the typed config. A nested *section* is returned as a plain
dict
114         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
115         # section model instead breaks it (a model has no .get), which bit every validator
on
116         # real runs. Scalar leaves are returned as-is.
117         def get(self, key: str, default: Any = None) -> Any:
118             _MISSING = object()
119             val: Any = _MISSING
120             if key in type(self).model_fields or hasattr(self, key):
121                 val = getattr(self, key, _MISSING)
122             if val is _MISSING:
123                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
124                 # discovered dynamically from the model fields, so a future top-level
section
```

```
                # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val

        def __getitem__(self, key: str) -> Any:
            val = self.get(key)
            if val is None and key not in type(self).model_fields:
                # Re-raise like a real dict for unknown top-level
                raise KeyError(key)
            return val


    # Convenience type alias for callers that want to annotate
    Config = AppConfig
```

7. user (2026-06-07T09:17:22.116Z)

```
      """Configuration loader and saver.

      Provides a single place to load validated AppConfig with built-in defaults
      merged with optional file overrides (config.json).

      This replaces scattered json.load + .get() patterns throughout the codebase.
      """

      from __future__ import annotations

      import json
      from pathlib import Path
      from typing import Any

      from .models import AppConfig

      DEFAULT_CONFIG_PATH = Path("config.json")


      def _get_builtin_defaults() -> dict[str, Any]:
          """Return a plain dict of the current built-in defaults.

          The AppConfig model is the single source of truth for defaults.
          """
          return AppConfig().model_dump(mode="json")


      def load_config(path: str | Path | None = None) -> AppConfig:
          """Load configuration with the following precedence:

          1. Built-in defaults defined in the Pydantic models.
          2. Values from the JSON file (if present and readable).
          3. (Future) environment / CLI overrides.

          Missing file or unreadable file ? returns a fully defaulted AppConfig.
          Unknown keys in the file are tolerated (extra="allow" on the model).
          """
          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

```

```
40          file_data: dict[str, Any] = {}
41          if cfg_path.exists():
42              try:
43                  with open(cfg_path, "r", encoding="utf-8") as f:
44                      file_data = json.load(f)
45              except Exception as exc:
46                  # Non-fatal: continue with defaults + whatever we could parse.
47                  # In production we might log here.
48                  print(f"[CONFIG WARNING] Failed to read {cfg_path}: {exc}. Using defaults +
partial data.")
49
50          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
51          merged = {**_get_builtin_defaults(), **file_data}
52
53          return AppConfig.model_validate(merged)
54
55
56      def save_default_config(path: str | Path | None = None) -> Path:
57          """Write a fresh default config.json.
58
59          Used by setup.py diagnostics and the --setup CLI path.
60          Overwrites if the file exists.
61          """
62          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
63          cfg = AppConfig()
64          cfg_path.parent.mkdir(parents=True, exist_ok=True)
65          cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
66          return cfg_path
67
68
69      # Backwards-compatible alias used by some older call sites during transition.
70      def get_default_config() -> dict[str, Any]:
71          """Return defaults as a plain dict (for transitional use only)."""
72          return _get_builtin_defaults()
73
```

8. assistant (2026-06-07T09:17:29.880Z)

Several discrepancies in the config section already. Let me note them and continue verifying. The CODE_MAP claims `__init__.py` re-exports include `Config` and `__all__` has exactly 6 items ? verified true. But the CODE_MAP says the `get` method docstring "bit every validator on real runs" ? that matches. Let me check `IndexingConfig` claims: CODE_MAP lists `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `chunk_duration=90.0`, `chunk_overlap=10.0` ? all match. The `tracknet` config has `conda_env='TrackNetV4'` (matches), `velocity_threshold=0.02` (matches).

But CODE_MAP §2.0 says `TrackNetConfig` has `repo_dir` ? yes. It does NOT mention `conda_sh`, `stage_in_wsl`, `wsl_stage_dir`, `timeout_sec` but that's omission, not error. Now let me verify the main.py, the segmenters base, and detectors.

9. user (2026-06-07T09:17:30.257Z)

```
1       import os
2       import sys
3       import argparse
4       import hashlib
5       import json
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10      from fastapi import FastAPI, HTTPException
11      from fastapi.middleware.cors import CORSMiddleware
12      from pydantic import BaseModel
13      from typing import List, Dict, Any, Optional
```

```
14
15      from fastapi.staticfiles import StaticFiles
16      from fastapi.responses import HTMLResponse
17      from backend.pipeline.validators.base import registry
18      from backend.infrastructure.database import Database
19      from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
20      from backend.pipeline.stitcher.compiler import VideoCompiler
21      from backend.config import load_config as load_app_config, AppConfig  # new typed config
22      from backend.reporting import emit_indexer_report
23
24      app = FastAPI(title="Sports Highlight Indexer API")
25
26      # Enable CORS for local web interface
27      app.add_middleware(
28          CORSMiddleware,
29          allow_origins=["*"],
30          allow_credentials=True,
31          allow_methods=["*"],
32          allow_headers=["*"],
33      )
34
35      # Serves static frontend files directly (now under api/ per approved structure)
36      os.makedirs("backend/api/static", exist_ok=True)
37      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
38
39      @app.get("/", response_class=HTMLResponse)
40      def serve_index():
41          index_path = "backend/api/static/index.html"
42          if os.path.exists(index_path):
43              with open(index_path, "r", encoding="utf-8") as f:
44                  return f.read()
45          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
46
47      # Global variables initialized at startup
48      db_instance: Optional[Database] = None
49      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
50
51      def get_file_md5(filepath: str) -> str:
52          """Calculates file hash to uniquely identify videos and prevent duplicate
indexes."""
53          hasher = hashlib.md5()
54          with open(filepath, "rb") as f:
55              # Read in chunk sizes of 64kb
56              for chunk in iter(lambda: f.read(65536), b""):
57                  hasher.update(chunk)
58          return hasher.hexdigest()
59
60      # Legacy thin wrapper for any remaining direct calls during transition.
61      # New code should import load_app_config / AppConfig from backend.config directly.
62      def load_config(config_path: str = "config.json") -> Dict[str, Any]:
63          cfg = load_app_config(config_path)
64          return cfg.model_dump(mode="json")
65
66      # --- API Models ---
67      class ProcessRequest(BaseModel):
68          video_path: str
69          player_pool: Optional[List[str]] = None
70          max_frames: Optional[int] = None
71          segmenter: Optional[str] = None
72
73      class QueryRequest(BaseModel):
74          video_id: str
75          server: Optional[str] = None
76          receiver: Optional[str] = None
```

```
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    @app.post("/api/process")
    def process_video(req: ProcessRequest):
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")

        video_id = get_file_md5(req.video_path)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        print(f"[API] Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.dict() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            print(f"[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                available = list(segmenter_registry.list_available().keys())
                raise HTTPException(
                    status_code=400,
                    detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
                )

            print(f"[API] Using segmenter: {seg_name}")
            segmenter_actual = seg_name
            segmenter = segmenter_cls(db_instance, global_config)
```

```
            segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            segmentation_ran = True

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.dict() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]

        # Run compiler
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")
```

```
193
194     # --- CLI Debug Utility & Orchestration ---
195     def run_cli_diagnose_clip(video_path: str, timestamp: float):
196         """CLI debugging utility to examine segment properties around a timestamp."""
197         print("=" * 60)
198         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
199         print("=" * 60)
200
201         cfg = load_config()  # legacy wrapper still works, returns dict for now
202
203         # 1. Validation check diagnostics
204         print("[DIAGNOSTIC] Running validation framework...")
205         results = registry.run_all(video_path, cfg)
206         for r in results:
207             status_symbol = "[PASS]" if r.passed else "[FAIL]"
208             print(f"  {status_symbol} {r.validator_name}: {r.message}")
209             if r.details:
210                 print(f"         Details: {r.details}")
211
212         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
213         print("-" * 60)
214         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
215         import cv2
216         cap = cv2.VideoCapture(video_path)
217         if not cap.isOpened():
218             print("[FAIL] OpenCV could not open video.")
219             return
220
221         fps = cap.get(cv2.CAP_PROP_FPS)
222         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
223
224         start_frame = max(0, int((timestamp - 3.0) * fps))
225         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
226
227         # Measure frame-by-frame changes in sample region
228         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
229         prev_gray = None
230         motions = []
231
232         for f in range(start_frame, end_frame):
233             ret, frame = cap.read()
234             if not ret:
235                 break
236             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
237             gray = cv2.GaussianBlur(gray, (21, 21), 0)
238
239             if prev_gray is not None:
240                 diff = cv2.absdiff(prev_gray, gray)
241                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
242                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
243                 motions.append(motion_ratio)
244             prev_gray = gray
245
246         cap.release()
247
248         if motions:
249             avg_motion = sum(motions) / len(motions)
250             # Support both legacy dict (from wrapper) and future direct model
251             if hasattr(cfg, "indexing"):
252                 threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
253             else:
254                 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
255             print(f"  Sample Frame Count: {len(motions)}")
```

```
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
        parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
        parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")

        args = parser.parse_args()

        if args.setup:
            # Invoke setup diagnostics
            import subprocess
            print("[INFO] Invoking setup.py...")
            subprocess.run([sys.executable, "setup.py"])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = args.output_dir
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

```

```
            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            compiler = VideoCompiler(out_dir)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # output_dir is a runtime concern (not stored in config.json).
        # We attach it to the config object for downstream compatibility during transition.
        # Downstream code that does dict access will still work via model_dump() if needed,
        # but new code should read cfg.output.output_dir after we extend the model.
        if not hasattr(global_config, "_runtime_overrides"):
            global_config._runtime_overrides = {}
        global_config._runtime_overrides["output_dir"] = args.output_dir

        # For code that still does global_config.get("output", {}) we provide a thin compat
layer
        # by using the model + overrides when necessary. For now we compute db_path
directly.
        os.makedirs(args.output_dir, exist_ok=True)

        db_name = getattr(getattr(global_config, "output", None), "db_name",
"sports_indexer.db")
        db_path = os.path.join(args.output_dir, db_name)
        db_instance = Database(db_path)

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            if not os.path.exists(args.video_path):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return

            video_id = get_file_md5(args.video_path)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])

            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
```

```
"default_segmenter", "motion")
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "="*40)
                print("VALIDATION SUMMARY")
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                segmenter = segmenter_cls(db_instance, global_config)
                segments = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
                segmentation_ran = True
                print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}")
            finally:
                # Always emit the per-video observability report. Never raises.
                paths = emit_indexer_report(
                    db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
                    val_results=val_results, val_context=val_context,
                    segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                    was_reingest=was_reingest, segmentation_ran=segmentation_ran,
                )
                if paths:
                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
            return

        # Start Local Server Mode
        if args.server or not args.video_path:
            print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
            uvicorn.run(app, host="127.0.0.1", port=args.port)

    if __name__ == "__main__":
        main()

```

10. assistant (2026-06-07T09:17:34.013Z)

main.py is well-described. Note `get_file_md5` uses 64KB (65536) chunk ? matches CODE_MAP. The `compile_highlights` `output_dir` fallback to "output" claim is correct (OutputConfig has no output_dir). Now let me verify the segmenters, detectors, and other core files.

11. user (2026-06-07T09:17:35.401Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
```

```
3       from backend.infrastructure.database import Database
4
5       class BasePlaySegmenter(ABC):
6           def __init__(self, db: Database, config: Dict[str, Any]):
7               self.db = db
8               self.config = config
9
10          @property
11          @abstractmethod
12          def name(self) -> str:
13              """Unique identifier for this segmenter."""
14              pass
15
16          @property
17          @abstractmethod
18          def description(self) -> str:
19              """Human-readable explanation of this segmenter."""
20              pass
21
22          @abstractmethod
23          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
24              """
25              Processes a video to detect play segments, populates SQLite database,
26              and returns list of segment representations.
27              """
28              pass
29
30      class SegmenterRegistry:
31          def __init__(self):
32              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
33
34          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
35              """Registers a new segmenter class under its name property."""
36              try:
37                  temp_inst = segmenter_cls(None, {})
38                  name = temp_inst.name
39              except Exception as e:
40                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
41              self._segmenters[name] = segmenter_cls
42
43          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
44              return self._segmenters.get(name)
45
46          def list_available(self) -> Dict[str, str]:
47              """Returns a dict of registered segmenter names and their descriptions."""
48              res = {}
49              for name, cls in self._segmenters.items():
50                  try:
51                      temp_inst = cls(None, {})
52                      desc = temp_inst.description
53                  except Exception:
54                      desc = "No description available."
55                  res[name] = desc
56              return res
57
58      segmenter_registry = SegmenterRegistry()
59
```

12. user (2026-06-07T09:17:35.882Z)

```
1       from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2       from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3       from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
```

```
4       from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6       from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7
8       __all__ = [
9           "segmenter_registry",
10          "BasePlaySegmenter",
11          "MotionPlaySegmenter",
12          "GeminiPlaySegmenter",
13          "HybridYoloSegmenter",
14          "TrackNetHybridSegmenter",
15          "WasbHybridSegmenter",
16      ]
17
```

13. user (2026-06-07T09:17:36.239Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22
23      logger = logging.getLogger(__name__)
24
25      # Windows path to the inference wrapper we stage into WSL.
26      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
27
28
29      @dataclass
30      class WasbConfig:
31          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
32          distro: str = "Ubuntu"
33          repo_dir: str = "~/models/WASB-SBDT"
34          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
35          conda_env: str = "wasb"
36          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
37          sport: str = "badminton"
38          wsl_stage_dir: str = "~/clips"
39          timeout_sec: int = 0  # 0 = no timeout
40
41          @classmethod
42          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
43              w = (idx_cfg or {}).get("wasb", {}) or {}
44              return cls(
45                  distro=w.get("wsl_distro", "Ubuntu"),
```

```
                repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
                conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=w.get("conda_env", "wasb"),
                weights_path=w.get("weights_path",
                                   "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
                sport=w.get("sport", "badminton"),
                wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(w.get("timeout_sec", 0)),
            )


    class WasbRunner:
        def __init__(self, cfg: WasbConfig):
            self.cfg = cfg

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
            logger.debug("WSL exec: %s", full)
            return subprocess.run(argv, capture_output=True, text=True,
                                  timeout=(self.cfg.timeout_sec or None))

        def healthcheck(self) -> tuple:
            """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
            cmd = (f"conda activate {self.cfg.conda_env} && "
                   f"python -c \"import torch; print('torch', torch.__version__, "
                   f"'cuda', torch.cuda.is_available())\"")
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? Windows + WSL2 required."
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
            return True, out

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            src_mnt = to_wsl_mnt_path(video_win_path)
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        def _stage_infer_script(self) -> bool:
            """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
            src_mnt = to_wsl_mnt_path(_INFER_WIN)
            cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
            res = self._wsl_bash(cmd)
            return res.returncode == 0 and "OK" in (res.stdout or "")

        def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
            """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
```

```
None."""
107             os.makedirs(output_win_dir, exist_ok=True)
108             base = os.path.basename(video_win_path)
109             stem = os.path.splitext(base)[0]
110             wsl_out_dir = to_wsl_mnt_path(output_win_dir)
111             expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
112
113             if not self._stage_infer_script():
114                 logger.error("Could not stage wasb_infer.py into WSL.")
115                 return None
116             staged_video = self._stage_video(video_win_path, base)
117             if staged_video is None:
118                 return None
119             frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
120
121             cmd = (
122                 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
123                 f"python wasb_infer.py "
124                 f"--video {shlex.quote(staged_video)} "
125                 f"--frames_out_dir {shlex.quote(frames_dir)} "
126                 f"--weights {shlex.quote(self.cfg.weights_path)} "
127                 f"--sport {shlex.quote(self.cfg.sport)} "
128                 f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}"
129             )
130             logger.info("Running WASB inference on %s ...", base)
131             try:
132                 res = self._wsl_bash(cmd)
133             except subprocess.TimeoutExpired:
134                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
135                 return None
136             if res.returncode != 0:
137                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
138                 return None
139             if not os.path.exists(expected_csv_win):
140                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
141                 return None
142             logger.info("WASB trajectory CSV: %s", expected_csv_win)
143             return expected_csv_win
144
145         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
146         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
147         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
148
```

14. assistant (2026-06-07T09:17:41.485Z)

The base segmenter `register()` builds `cls(None, {})` matching CODE_MAP's claim. `wasb_runner.py` confirms WasbConfig has `weights_path` with real default, the staticmethod rebinding, `_INFER_WIN`, hard-named `<stem>_wasb.csv`, `_wsl_bash` is duplicated. All correct. Let me verify tracknet_runner, the hybrids, and rally_gate.

15. user (2026-06-07T09:17:42.433Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
```

```
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      logger = logging.getLogger(__name__)
26
27
28      @dataclass
29      class TrackNetConfig:
30          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
31          distro: str = "Ubuntu"
32          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
33          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
34          conda_env: str = "TrackNetV4"
35          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
36          queue_length: int = 5
37          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
38          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
39          timeout_sec: int = 0                            # 0 = no timeout
40
41          @classmethod
42          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
43              tn = (idx_cfg or {}).get("tracknet", {}) or {}
44              return cls(
45                  distro=tn.get("wsl_distro", "Ubuntu"),
46                  repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
47                  conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
48                  conda_env=tn.get("conda_env", "TrackNetV4"),
49                  weights_path=tn.get("weights_path", ""),
50                  queue_length=int(tn.get("queue_length", 5)),
51                  stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
52                  wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
53                  timeout_sec=int(tn.get("timeout_sec", 0)),
54              )
55
56
57      def to_wsl_mnt_path(win_path: str) -> str:
58          """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
59
60          Already-POSIX paths (starting with / or ~) are returned unchanged so the
61          same helper is safe to call on either kind of path.
62          """
63          p = str(win_path)
64          if p.startswith("/") or p.startswith("~"):
65              return p
66          p = p.replace("\\", "/")
67          if len(p) >= 2 and p[1] == ":":
68              drive = p[0].lower()
69              return f"/mnt/{drive}{p[2:]}"
```

```
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner:
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows."""

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        # ------------------------------------------------------------------ WSL exec

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            """Run a bash command string inside the configured WSL distro."""
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
            logger.debug("WSL exec: %s", full)
            return subprocess.run(
                argv, capture_output=True, text=True,
                timeout=(self.cfg.timeout_sec or None),
            )

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
```

```
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            os.makedirs(output_win_dir, exist_ok=True)
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, base)
                if staged is None:
                    return None
                wsl_video = staged
            else:
                wsl_video = to_wsl_mnt_path(video_win_path)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {shlex.quote(wsl_video)} "
                f"--model_weights {shlex.quote(self.cfg.weights_path)} "
                f"--output_dir {shlex.quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
```

```
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------ CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            # Compute per-frame normalised velocity from the last *visible* point.
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
```

```
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p

            if not active:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            windows = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
                    group = [af]
            windows.append(_finalize(group))

            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

```

16. user (2026-06-07T09:17:42.831Z)

```
"""Rally-start gate: suppress between-point false fires using shuttle net-crossings.

WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
the opponent for service after losing a point, that single trajectory looks like a
rally. The discriminator is structural, not duration-based: a real rally has the
shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
single service feed crosses ~once.

This module is camera-agnostic and needs NO new model ? it works on the trajectory
we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
axis with the larger shuttle spread) and the net position (median shuttle coord on
that axis ? the shuttle clusters at the two ends and crosses the net region often,
so the median sits near the net), then counts sign changes of (coord - net) across
the visible points inside a candidate window, with a deadband to ignore jitter.

Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
windows from trajectory_to_action_windows, or to fold into it later.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import List, Sequence, Tuple

Interval = Tuple[float, float]


```

```
    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
```

```
List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable)."""
        axis, net, span = estimate_play_axis(points)
        deadband = deadband_frac * span
        out: List[GatedWindow] = []
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
            vis = sum(1 for p in wp if p.visible)
            out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                                   kept=c >= min_crossings))
        return out


    def apply_gate(points, windows: List[Interval], fps: float,
                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
        """Convenience: return only the windows that pass the crossing gate."""
        return [(g.start, g.end) for g in
                gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

17. assistant (2026-06-07T09:17:47.912Z)

Both files match the CODE_MAP. Note: CODE_MAP §2.2 says TrackNetConfig `from_indexing_cfg` has "key drift `wsl_distro` JSON -> `distro` field" ? confirmed (line 45: `distro=tn.get("wsl_distro", ...)`). The `parse_trajectory_csv` and `trajectory_to_action_windows` are `@staticmethod` ? confirmed. Now let me verify the eval subsystem, infrastructure/database, reporting, and validators.

18. user (2026-06-07T09:17:48.863Z)

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
5       wrapper over machinery that already exists:
6
7           ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8           predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9           comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
10          baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12      A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13      baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14      later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15      independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16      """
17      import os
18      import json
19      import logging
20      from dataclasses import dataclass, asdict
21      from typing import Dict, List, Optional
22
23      from backend.infrastructure.database import Database
24      from backend.eval import metrics
25      from backend.eval.harness import get_predictions
26      from backend.annotations import annotation_registry
27
28      logger = logging.getLogger(__name__)
29
30      DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
31      DEFAULT_TOLERANCE = 0.05
32
```

```

    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Use this to (re)baseline after a *legitimate* production improvement, so old
        baselines don't raise false regressions. Stored as plain JSON for easy diffing.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)


    # ---------------------------------------------------------------- the runner

    def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
                stage: str = "final", iou_threshold: float = 0.5,
                detector: Optional[str] = None,
                tolerance: float = DEFAULT_TOLERANCE,
                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
        """Score stored predictions for one video against ground truth and compare to
baseline.

        `ground_truth` is a list of (start, end) intervals ? typically obtained from an
        AnnotationProvider (see `regress_with_provider`). A regression is flagged when
```

```
        precision OR recall falls more than `tolerance` below the snapshotted baseline.
        If no baseline exists yet, `regressed` is False (nothing to compare to) and the
        caller can choose to snapshot the current numbers via `save_baseline`.
        """
        preds = get_predictions(db, video_id, stage, detector=detector)
        s = metrics.score(preds, ground_truth, iou_threshold)

        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
            if s.precision < base_p - tolerance:
                regressed = True
                reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
        )


    def regress_with_provider(db: Database, video_id: str, video_ref: str,
                              tier: str = "file", stage: str = "final",
                              iou_threshold: float = 0.5,
                              detector: Optional[str] = None,
                              tolerance: float = DEFAULT_TOLERANCE,
                              baseline_path: str = DEFAULT_BASELINE_PATH,
                              guideline: Optional[str] = None,
                              sport: str = "badminton",
                              provider_kwargs: Optional[dict] = None) -> RegressionResult:
        """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.

        `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
        once those providers land). `video_ref` is whatever that provider expects (a CSV
        path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
        predictions in the DB (the file MD5).
        """
        provider = annotation_registry.get(tier, **(provider_kwargs or {}))
        gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
        return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)


    def format_result(r: RegressionResult) -> str:
        verdict = "REGRESSED" if r.regressed else "ok"
        head = (f"[{verdict}] {r.video_id} ({r.stage}) "
                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
        if r.baseline_precision is not None:
            head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
        return head + "".join(f"\n   - {why}" for why in r.reasons)
```

157

19. user (2026-06-07T09:17:49.019Z)

```
"""Score the pipeline's detected rallies against ground-truth annotations.

Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
so scoring is fast, deterministic, and free. Two prediction "stages" can be
scored:

  - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
    the shuttle/person detector found, before AI confirmation.
  - ``final``      ? confirmed play segments (``play_segments`` table). After the
    AI handoff trimmed/validated boundaries.

Comparing the two tells you whether the weak link is *detection* (candidates
already miss rallies) or *segmentation* (candidates catch them but final is
wrong) ? see docs/EVALUATION.md.
"""

import logging
from dataclasses import dataclass, field
from typing import List, Dict, Optional

from backend.infrastructure.database import Database
from backend.eval import metrics
from backend.eval.metrics import Interval, Score, MatchResult

logger = logging.getLogger(__name__)

STAGES = ("candidates", "final")


@dataclass
class StageReport:
    stage: str
    pred_intervals: List[Interval]
    score: Score
    match_result: MatchResult
    pr_curve: List[Score] = field(default_factory=list)


@dataclass
class EvalReport:
    video_id: str
    iou_threshold: float
    gt_intervals: List[Interval]
    stages: Dict[str, StageReport] = field(default_factory=dict)


def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
    """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
    ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
    ivs.sort(key=lambda iv: iv[0])
    return ivs


def get_predictions(db: Database, video_id: str, stage: str,
                    detector: Optional[str] = None) -> List[Interval]:
    """Fetch predicted rally intervals for one stage from the DB."""
    if stage == "candidates":
        rows = db.query_candidate_segments(video_id, detector=detector)
    elif stage == "final":
        rows = db.query_play_segments(video_id, {})
    else:
```

```
            raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
        return _rows_to_intervals(rows)


    def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
                 stages: List[str], iou_threshold: float = 0.5,
                 detector: Optional[str] = None,
                 with_pr_curve: bool = False) -> EvalReport:
        """Score the requested stages for one video against ground truth."""
        report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
                            gt_intervals=gt_intervals)
        for stage in stages:
            preds = get_predictions(db, video_id, stage, detector=detector)
            mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
            s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
            curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
            report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
                                               score=s, match_result=mr, pr_curve=curve)
        return report


    # ---------------------------------------------------------------------- reporting

    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    def report_to_dict(report: EvalReport) -> dict:
        """JSON-serialisable view of an EvalReport."""
        out = {
            "video_id": report.video_id,
            "iou_threshold": report.iou_threshold,
            "num_ground_truth": len(report.gt_intervals),
            "stages": {},
        }
        for stage, sr in report.stages.items():
            out["stages"][stage] = {
                "score": sr.score.as_dict(),
                "pr_curve": [s.as_dict() for s in sr.pr_curve],
            }
        return out


    def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
        """Human-readable console report for one video."""
        lines = [
            "=" * 64,
            f" Rally evaluation ? video '{report.video_id}'",
            f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
            "=" * 64,
            f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
            f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
            "-" * 64,
        ]
        for stage in report.stages:
            s = report.stages[stage].score
            lines.append(
                f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
                f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
                f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
```

```
            )
        lines.append("=" * 64)

        if show_failures:
            for stage, sr in report.stages.items():
                misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
                fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
                lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
                for iv in misses:
                    lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
                lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
                for iv in fps:
                    lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")

        return "\n".join(lines)

```

20. assistant (2026-06-07T09:17:53.552Z)

Both verified. regression.py imports `annotation_registry` from `backend.annotations` (line 26) NOT `backend.eval.annotations` ? confirmed CODE_MAP claim. `STAGES=('candidates','final')`, `get_predictions` raises ValueError for unknown stage ? confirmed. Now let me verify the database, reporting/__init__ and harvest, and the validators.

21. user (2026-06-07T09:17:54.970Z)

```
import sqlite3
import json
import os
import logging
from typing import List, Dict, Any, Optional

logger = logging.getLogger(__name__)

class Database:
    def __init__(self, db_path: str):
        self.db_path = db_path
        self._init_db()

    def _get_connection(self):
        conn = sqlite3.connect(self.db_path)
        conn.row_factory = sqlite3.Row
        # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
        conn.execute("PRAGMA foreign_keys = ON")
        return conn

    def _init_db(self):
        """Initializes tables for videos, play_segments, and events."""
        with self._get_connection() as conn:
            cursor = conn.cursor()

            # Videos Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    id TEXT PRIMARY KEY,
                    filepath TEXT NOT NULL,
                    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    status TEXT NOT NULL,
                    validation_results TEXT
                )
            """)

```

```
                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
```

```
97                      """)
98                      cursor.execute("PRAGMA user_version = 1")
99                  # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 2
100
101                 conn.commit()
102
103         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
104             try:
105                 with self._get_connection() as conn:
106                     conn.cursor().execute(
107                         "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
108                         (video_id, filepath, status, json.dumps(validation_results))
109                     )
110                     conn.commit()
111                 return True
112             except Exception as e:
113                 logger.error(f"Failed to add video: {e}")
114                 return False
115
116         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
117             try:
118                 with self._get_connection() as conn:
119                     cursor = conn.cursor()
120                     cursor.execute(
121                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
122                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
123                     )
124                     conn.commit()
125                     return cursor.lastrowid
126             except Exception as e:
127                 logger.error(f"Failed to add play segment: {e}")
128                 return None
129
130         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
131             try:
132                 with self._get_connection() as conn:
133                     conn.cursor().execute(
134                         "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
135                         (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
136                     )
137                     conn.commit()
138                 return True
139             except Exception as e:
140                 logger.error(f"Failed to add event: {e}")
141                 return False
142
143         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
144                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
145                                   stats: Optional[Dict[str, Any]] = None,
146                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
147             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
148             detector-specific dicts stored as JSON. Returns the new candidate id, or
```

```
None."""
            try:
                with self._get_connection() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._get_connection() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._get_connection() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
```

```
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._get_connection() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

```

22. user (2026-06-07T09:17:55.135Z)

```
1       """Per-video observability reports for the indexer.
2
3       `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4       no-op and the pipeline runs exactly as before. The single entrypoint
5       `emit_indexer_report(...)` is designed to be called from a `finally` block and
6       NEVER raises ? emitting a report must not be able to break a processing run.
7       """
8       from __future__ import annotations
9
10      import os
11      from typing import Any, Dict, List, Optional
12
13      try:  # soft dependency ? never hard-fail the pipeline on a reporting import
14          import obsreport  # noqa: F401
15          from . import harvest
16
17          OBSREPORT_AVAILABLE = True
18      except Exception:  # pragma: no cover - exercised only when obsreport is absent
19          OBSREPORT_AVAILABLE = False
20
21
22      def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
23          return bool((config or {}).get("report", {}).get("enabled", True))
24
25
26      def emit_indexer_report(
27          *,
28          db,
29          video_id: str,
30          video_path: Optional[str],
31          config: Optional[Dict[str, Any]],
32          val_results: List[Any],
33          val_context: Optional[Dict[str, Any]] = None,
34          segmenter_requested: Optional[str] = None,
35          segmenter_actual: Optional[str] = None,
36          was_reingest: bool = False,
37          segmentation_ran: bool = True,
38      ) -> Optional[Dict[str, str]]:
39          """Harvest + write the report. Returns the written file paths, or None if
40          reporting is unavailable/disabled/failed. Never raises."""
41          config = config or {}
42          # Accept either a plain dict or a typed config object (e.g.
backend.config.AppConfig).
43          # Normalize to a plain nested dict so harvest's dict access works uniformly.
44          if hasattr(config, "model_dump"):
45              try:
46                  config = config.model_dump(mode="json")
47              except Exception:
48                  config = {}
49          if not OBSREPORT_AVAILABLE or not report_enabled(config):
50              return None
51          try:
52              play_segments = db.query_play_segments(video_id, {}) if db is not None else []
53              candidates = db.query_candidate_segments(video_id) if db is not None else []
54              rec = harvest.record_run(
55                  video_id=video_id,
56                  video_path=video_path,
57                  config=config,
58                  val_results=val_results,
59                  val_context=val_context,
60                  play_segments=play_segments,
61                  candidates=candidates,
62                  segmenter_requested=segmenter_requested,
```

```
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            output_dir = (config.get("report") or {}).get("output_dir")
            res = rec.write(output_dir=output_dir)
            return res.paths or None
        except Exception:  # pragma: no cover - defensive; reporting must never break a run
            return None

```

23. assistant (2026-06-07T09:18:01.540Z)

Database matches. CODE_MAP §2.7: 4 tables, migration ladder, `_get_connection` re-issues PRAGMA, `INSERT OR REPLACE`, events column is `type` not `event_type`, query truthiness filters, write helpers swallow exceptions ? all confirmed. The candidate row has `EXPECTED_CANDIDATE_COLUMNS` mentioned in test map; let me check that test references something real but the DB file doesn't define that constant (it would be in the test). That's fine.

One note: CODE_MAP §2.8 says `emit_indexer_report` writes via `rec.write(output_dir=config["report"]["output_dir"])`. The actual code uses `(config.get("report") or {}).get("output_dir")` ? close enough, descriptive. Now let me verify the validators and a few remaining eval/segmenter files.

24. user (2026-06-07T09:18:03.385Z)

```
from backend.pipeline.validators.base import registry, VideoValidator, ValidationResult
from backend.pipeline.validators.format import FormatValidator
from backend.pipeline.validators.resolution_fps import ResolutionFPSValidator
from backend.pipeline.validators.pts_continuity import PTSContinuityValidator
from backend.pipeline.validators.scene_cut import SceneCutDensityValidator
from backend.pipeline.validators.blur import BlurValidator
from backend.pipeline.validators.relevance import RelevanceValidator

# Register all default validators in sequence (dependencies first)
registry.register(FormatValidator())
registry.register(ResolutionFPSValidator())
registry.register(PTSContinuityValidator())
registry.register(SceneCutDensityValidator())
registry.register(BlurValidator())
registry.register(RelevanceValidator())

__all__ = ["registry", "VideoValidator", "ValidationResult"]

```

25. user (2026-06-07T09:18:03.420Z)

```
"""Build an observability report for one indexer run.

Pure-ish harvest: given the validation results/context, the persisted segments,
and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
*footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
so this module only records facts faithfully ? it does not decide warnings.

Imported only after `backend.reporting` confirms obsreport is installed, so it may
import obsreport freely.
"""
from __future__ import annotations

import os
from pathlib import Path
from typing import Any, Dict, List, Optional

import obsreport
```

```
18
19      _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
20      _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
21
22
23      def load_spec() -> "obsreport.ReportSpec":
24          return obsreport.load_spec(_SPEC_PATH)
25
26
27      # --- media metadata ----------------------------------------------------
28      def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
29          """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
30          distinguishes a real probe from the absence of one (the fallback footgun)."""
31          meta: Dict[str, Any] = {"fps_source": "unknown"}
32          if video_path and os.path.exists(video_path):
33              try:
34                  meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
35              except OSError:
36                  pass
37
38          ffmeta = (val_context or {}).get("ffprobe_meta") or {}
39          streams = ffmeta.get("streams") or []
40          fmt = ffmeta.get("format") or {}
41          if streams:
42              s = streams[0]
43              w, h = s.get("width"), s.get("height")
44              meta["width"], meta["height"] = w, h
45              if w and h:
46                  meta["resolution"] = f"{w}x{h}"
47              fps_str = s.get("r_frame_rate", "0/1")
48              try:
49                  num, den = (int(x) for x in fps_str.split("/"))
50                  if den:
51                      meta["fps"] = round(num / den, 2)
52                      meta["fps_source"] = "probed"
53              except (ValueError, ZeroDivisionError):
54                  pass
55              if s.get("codec_name"):
56                  meta["container"] = fmt.get("format_name") or s.get("codec_name")
57          if fmt.get("duration"):
58              try:
59                  meta["duration_s"] = round(float(fmt["duration"]), 1)
60              except (ValueError, TypeError):
61                  pass
62          if fmt.get("format_name"):
63              meta["container"] = fmt.get("format_name")
64          return meta
65
66
67      # --- stage builders ----------------------------------------------------
68      def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
69          with rec.stage("validation") as st:
70              total = len(val_results)
71              passed = sum(1 for r in val_results if getattr(r, "passed", False))
72              st.add_metric("checks_passed", passed)
73              st.add_metric("checks_total", total)
74              failures = [r for r in val_results if not getattr(r, "passed", True)]
75              for r in failures:
76                  st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
77              st.status = "error" if failures else ("ok" if total else "not_run")
78
79
80      def _segmentation_stage(
```

```
        rec: "obsreport.RunRecorder",
        play_segments: List[Dict[str, Any]],
        candidates: List[Dict[str, Any]],
        segmenter_requested: Optional[str],
        segmenter_actual: Optional[str],
        config_default: Optional[str],
        was_reingest: bool,
        ran: bool,
    ) -> None:
        with rec.stage("segmentation") as st:
            if not ran:
                st.status = "not_run"
            seg_count = len(play_segments)
            total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
            detector_tag = None
            if play_segments:
                detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
            st.add_metric("candidate_windows", len(candidates))
            st.add_metric("segment_count", seg_count, audience="both")
            st.add_metric("total_play_s", total_play, unit="s")
            # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
            # pick it, and it differs from config's default (the genuine surprise case).
            # An explicit --segmenter override or a halted run is not a divergence.
            matches_default = True
            if ran and segmenter_actual and config_default:
                matches_default = (segmenter_actual == config_default)
            st.details.update({
                "segmenter_requested": segmenter_requested,
                "segmenter_actual": segmenter_actual,
                "config_default": config_default,
                "segmenter_explicitly_requested": segmenter_requested is not None,
                "matches_config_default": matches_default,
                "was_reingest": was_reingest,
                "detector": detector_tag,
            })
            # Surface metadata trustworthiness at the data level (not just in a warning):
            # the motion baseline fabricates per-segment metadata; everything else measures
it.
            if ran and segmenter_actual:
                st.details["metadata_source"] = (
                    "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
                    else "measured (detector/AI)"
                )


    def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
                  play_segments: List[Dict[str, Any]], ran: bool) -> None:
        """Only meaningful for AI-based segmenters. Records whether the provider key was
        present ? the signal the silent_provider_noop rule keys on."""
        ai_based = segmenter_actual in _AI_SEGMENTERS
        provider = (config.get("ai_provider") or "gemini")
        key_env = f"{provider.upper()}_API_KEY"
        with rec.stage("ai_handoff") as st:
            if not ran or not ai_based:
                st.status = "not_run" if not ran else "skipped"
            st.details.update({
                "ai_based": ai_based,
                "provider": provider if ai_based else None,
                "model": config.get("ai_model") if ai_based else None,
                "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
            })
            if ai_based:
                st.add_metric("confirmed_segments", len(play_segments))

```

```
144
145     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
146                     video_meta: Dict[str, Any]) -> None:
147         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
148         coverage = None
149         dur = video_meta.get("duration_s")
150         if dur:
151             coverage = round(100.0 * total / dur, 1)
152         notes = []
153         if not play_segments:
154             notes.append("No highlights were produced for this video.")
155         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
156                            coverage_pct=coverage, notes=notes)
157
158
159     # --- top-level ------------------------------------------------------------
160     def record_run(
161         *,
162         video_id: str,
163         video_path: Optional[str],
164         config: Dict[str, Any],
165         val_results: List[Any],
166         val_context: Optional[Dict[str, Any]],
167         play_segments: List[Dict[str, Any]],
168         candidates: List[Dict[str, Any]],
169         segmenter_requested: Optional[str] = None,
170         segmenter_actual: Optional[str] = None,
171         was_reingest: bool = False,
172         segmentation_ran: bool = True,
173         tool_version: Optional[str] = None,
174         spec: Optional["obsreport.ReportSpec"] = None,
175         **recorder_kwargs: Any,
176     ) -> "obsreport.RunRecorder":
177         """Assemble a finalized recorder for an indexer run (does NOT write)."""
178         rec = obsreport.RunRecorder(
179             "badminton-highlight-indexer",
180             tool_version=tool_version,
181             video_id=video_id,
182             video_id_kind="file_md5",
183             video_path=video_path,
184             spec=spec if spec is not None else load_spec(),
185             **recorder_kwargs,
186         )
187         vm = video_meta_from_context(val_context, video_path)
188         # If we never probed an fps but segmentation still ran, the segmenter must have
189         # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
190         # AND show the assumed numeric value so the report reads "30 (fallback)" not "?
(fallback)".
191         if vm.get("fps_source") == "unknown" and segmentation_ran:
192             vm["fps_source"] = "fallback"
193             if vm.get("fps") is None:
194                 vm["fps"] = 30.0  # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
195         rec.set_video_meta(**vm)
196
197         _validation_stage(rec, val_results)
198         config_default = (config.get("indexing") or {}).get("default_segmenter")
199         _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
200                             segmenter_actual, config_default, was_reingest,
segmentation_ran)
201         _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran)
202         _highlights(rec, play_segments, vm)
203
204         rec.finalize()
205         return rec
```

26. assistant (2026-06-07T09:18:08.972Z)

Validators __init__ confirms exec order. harvest.py confirms `_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}`, `record_run`, `_SPEC_PATH`/`load_spec`, `video_meta_from_context`, all stage builders, the fps fallback logic, `config_default = config["indexing"]["default_segmenter"]`. All match.

Now let me verify the remaining items: the segmenters (yolo_hybrid, wasb_hybrid, gemini, motion), wasb_infer, the annotations subsystem, providers, sports, and a few eval files. Let me check symbol presence efficiently with grep across multiple files.

27. user (2026-06-07T09:18:10.973Z)

```
32:_PERSON_CLASS_ID = 1
36:_DETECTOR_FACTORIES = {
41:_DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
52:class HybridYoloSegmenter(BasePlaySegmenter):
67:    def _lazy_load_model(self):
81:        model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
82:        if model_name not in _DETECTOR_FACTORIES:
83:            logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
84:            model_name = _DEFAULT_DETECTOR
86:        builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
93:    def _make_tracker(self, fps: float):
125:        mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
142:    def _extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
178:        tracker = self._make_tracker(effective_fps)
264:        self._lazy_load_model()
321:            "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
376:                windows = self._extract_action_windows_from_chunk(
401:                windows = self._extract_action_windows_from_chunk(
```

28. user (2026-06-07T09:18:12.427Z)

```
backend\pipeline\segmenters\base.py:12:    def name(self) -> str:
backend\pipeline\segmenters\gemini.py:18:    def name(self) -> str:
backend\pipeline\segmenters\gemini.py:401:            # Extract shot_count from Gemini or
compute estimate if not present
backend\pipeline\segmenters\gemini.py:402:            shot_count =
segment.get("shuttle_exchange_count")
backend\pipeline\segmenters\gemini.py:403:            if shot_count is None:
backend\pipeline\segmenters\gemini.py:404:                shot_count = max(3, int(dur / 0.8))  #
1 shot every 0.8 seconds as fallback estimate
backend\pipeline\segmenters\gemini.py:407:                    shot_count = int(shot_count)
backend\pipeline\segmenters\gemini.py:409:                    shot_count = max(3, int(dur /
0.8))
backend\pipeline\segmenters\gemini.py:413:                "shot_count": shot_count,
backend\pipeline\segmenters\gemini.py:414:                "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
backend\pipeline\segmenters\motion.py:11:    def name(self) -> str:
backend\pipeline\segmenters\motion.py:166:                "shot_count": random.randint(4, 25),
backend\pipeline\segmenters\tracknet_hybrid.py:39:class
TrackNetHybridSegmenter(BasePlaySegmenter):
backend\pipeline\segmenters\tracknet_hybrid.py:41:    def name(self) -> str:
backend\pipeline\segmenters\tracknet_hybrid.py:50:    def _probe_fps_width(video_path: str) ->
tuple:
backend\pipeline\segmenters\tracknet_hybrid.py:91:        fps, frame_width =
self._probe_fps_width(video_path)
backend\pipeline\segmenters\tracknet_hybrid.py:224:                shot_count =
segment.get("shuttle_exchange_count")
backend\pipeline\segmenters\tracknet_hybrid.py:225:                if shot_count is None:
```

```
backend\pipeline\segmenters\tracknet_hybrid.py:226:                    shot_count = max(3,
int(dur / 0.8))
backend\pipeline\segmenters\tracknet_hybrid.py:229:                        shot_count =
int(shot_count)
backend\pipeline\segmenters\tracknet_hybrid.py:231:                        shot_count = max(3,
int(dur / 0.8))
backend\pipeline\segmenters\tracknet_hybrid.py:233:                    "shot_count": shot_count,
backend\pipeline\segmenters\tracknet_hybrid.py:234:                    "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
backend\pipeline\segmenters\wasb_hybrid.py:37:class WasbHybridSegmenter(BasePlaySegmenter):
backend\pipeline\segmenters\wasb_hybrid.py:39:    def name(self) -> str:
backend\pipeline\segmenters\wasb_hybrid.py:48:    def _probe_fps_width(video_path: str) ->
tuple:
backend\pipeline\segmenters\wasb_hybrid.py:83:        fps, frame_width =
self._probe_fps_width(video_path)
backend\pipeline\segmenters\wasb_hybrid.py:203:                shot_count =
segment.get("shuttle_exchange_count")
backend\pipeline\segmenters\wasb_hybrid.py:205:                    shot_count = int(shot_count)
backend\pipeline\segmenters\wasb_hybrid.py:207:                    shot_count = max(3, int(dur /
0.8))
backend\pipeline\segmenters\wasb_hybrid.py:208:                metadata = {"shot_count":
shot_count,
backend\pipeline\segmenters\wasb_hybrid.py:209:
"shot_count_confidence": segment.get("shot_count_confidence", "Unknown"),
backend\pipeline\segmenters\yolo_hybrid.py:59:    def name(self) -> str:
backend\pipeline\segmenters\yolo_hybrid.py:136:            if tid is None:
backend\pipeline\segmenters\yolo_hybrid.py:539:                shot_count =
segment.get("shuttle_exchange_count")
backend\pipeline\segmenters\yolo_hybrid.py:540:                if shot_count is None: shot_count
= max(3, int(dur / 0.8))
backend\pipeline\segmenters\yolo_hybrid.py:542:                    try: shot_count =
int(shot_count)
backend\pipeline\segmenters\yolo_hybrid.py:543:                    except (ValueError,
TypeError): shot_count = max(3, int(dur / 0.8))
backend\pipeline\segmenters\yolo_hybrid.py:546:                    "shot_count": shot_count,
backend\pipeline\segmenters\yolo_hybrid.py:547:                    "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
```

29. assistant (2026-06-07T09:18:17.269Z)

All those symbols verified. The CODE_MAP's claim about the "divergent shot_count" drift between wasb (`int(shot_count)` without `is None` short-circuit at line 203-207) vs tracknet (`is None` check at line 224-226) is confirmed accurate. The `_PERSON_CLASS_ID=1` confirmed. Now let me verify the names of the segmenters and check wasb_infer symbols, plus the remaining eval modules and annotations/providers/sports.

30. user (2026-06-07T09:18:19.699Z)

```
backend/pipeline/segmenters/wasb_hybrid.py:40:        return "wasb_hybrid"
backend/pipeline/segmenters/tracknet_hybrid.py:42:        return "tracknet_hybrid"
backend/pipeline/segmenters/yolo_hybrid.py:60:        return "yolo_hybrid"
backend/pipeline/segmenters/gemini.py:19:        return "gemini"
backend/pipeline/segmenters/gemini.py:23:        return "High-precision AI-powered semantic play
segment detection using Google Gemini Multimodal API."
backend/pipeline/segmenters/motion.py:12:        return "motion"
backend/pipeline/segmenters/motion.py:16:        return "Traditional CV pixel-motion heuristics
tracking frame differences."
```

31. user (2026-06-07T09:18:21.338Z)

```
47:CACHE_VERSION = 1
48:_DET_FILE = "det_raw.jsonl"
49:_MANIFEST_FILE = "manifest.json"
52:def _build_cfg(weights: str, sport: str):
```

```
77:    return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
80:def default_cache_dir(out_csv: str) -> str:
87:def _atomic_write_text(path: str, text: str) -> None:
103:def _dets_to_tracker(plain_list):
113:    _atomic_write_text(mpath, json.dumps(manifest, indent=2))
127:def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
130:    if not manifest or manifest.get("version") != CACHE_VERSION:
149:def load_and_clean_cache(cache_dir: str):
177:        _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
189:def _atomic_write_trajectory(out_csv: str, rows) -> None:
204:def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
217:    cfg = _build_cfg(weights, sport)
238:        cache_dir = default_cache_dir(out_csv)
242:    resume = manifest_compatible(
247:        windows_done, det_by_fid = load_and_clean_cache(cache_dir)
257:        "version": CACHE_VERSION,
336:        r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
339:    _atomic_write_trajectory(out_csv, rows)
346:def extract_frames(video_path: str, frames_dir: str) -> str:
```

32. assistant (2026-06-07T09:18:26.201Z)

All segmenter names confirmed (`wasb_hybrid`, `tracknet_hybrid`, `yolo_hybrid`, `gemini`, `motion`). wasb_infer.py has all claimed symbols. Note the CODE_MAP lists `::run` core, `_INFER_WIN` (that's in wasb_runner, not wasb_infer ? correct). Now let me verify the remaining eval modules, annotations, providers, sports, stitcher, tools, utils in bulk.

33. user (2026-06-07T09:18:28.239Z)

```
backend\eval\metrics.py:15:def interval_iou(a: Interval, b: Interval) -> float:
backend\eval\metrics.py:33:class Match:
backend\eval\metrics.py:41:class MatchResult:
backend\eval\metrics.py:48:def match_intervals(preds: List[Interval], gts: List[Interval],
backend\eval\metrics.py:83:class Score:
backend\eval\metrics.py:102:def score(preds: List[Interval], gts: List[Interval],
backend\eval\metrics.py:136:def pr_curve(preds: List[Interval], gts: List[Interval],
backend\eval\batch.py:19:_FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
backend\eval\batch.py:22:def resolve_video_path(local_path: Optional[str], videos_root: str) ->
Optional[str]:
backend\eval\batch.py:40:def default_stages_for(segmenter: str, requested: str = "auto") ->
List[str]:
backend\eval\batch.py:52:    if segmenter in _FINAL_ONLY_SEGMENTERS:
backend\eval\batch.py:61:def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str,
dict]:
backend\eval\batch.py:100:def format_aggregate(agg: Dict[str, dict]) -> str:
```

34. user (2026-06-07T09:18:30.185Z)

```
backend\eval\distill_local.py:45:BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's production-ish
windowing, no learning
backend\eval\distill_local.py:74:    vt, mg, mwd = BASELINE_LOCAL
backend\eval\calibrate_wasb.py:31:BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig /
trajectory_to_action_windows defaults
backend\eval\calibrate_wasb.py:34:def load_golden_gts(csv_path: str) -> List[Interval]:
backend\eval\calibrate_wasb.py:52:def make_predict_fn(trajectory_csv: str, fps: float,
frame_width: float) -> Callable[[Config], List[Interval]]:
backend\eval\calibrate_wasb.py:67:def default_grid() -> List[Config]:
backend\eval\calibrate_wasb.py:82:    base = evaluate(predict_fn(BASELINE), gts, iou)
backend\eval\calibrate_wasb.py:84:    fit = improvement_report(predict_fn, BASELINE, best_cfg,
gts, iou)   # OPTIMISTIC (fit on full GT)
backend\eval\calibrate_wasb.py:89:    print(f"baseline cfg {BASELINE}:
precision={base.precision:.3f} recall={base.recall:.3f} "
backend\eval\calibrate_local.py:35:LocalConfig = Tuple[float, float, float, int]
backend\eval\calibrate_local.py:36:BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0)   # today's
```

```
local default (windowing, no gate)
backend\eval\calibrate_local.py:39:def make_local_predict_fn(trajectory_csv: str, fps: float,
frame_width: float) -> Callable[[LocalConfig], list]:
backend\eval\calibrate_local.py:44:    def predict_fn(cfg: LocalConfig) -> List[Tuple[float,
float]]:
backend\eval\calibrate_local.py:58:def local_grid() -> List[LocalConfig]:
backend\eval\calibrate_local.py:66:def build_videos(specs: List[Dict]) -> List[Dict]:
backend\eval\calibrate_local.py:87:        imp = improvement_report(v["predict_fn"],
BASELINE_LOCAL, best_cfg, v["gts"], iou)
backend\eval\calibrate_local.py:90:        print(f"  baseline {BASELINE_LOCAL}:
P={b['precision']:.3f} R={b['recall']:.3f} F1={b['f1']:.3f}")
backend\eval\calibrate_local.py:98:    base_held = [evaluate(v["predict_fn"](BASELINE_LOCAL),
v["gts"], iou) for v in videos]
backend\eval\export_wasb_segments.py:27:from backend.eval.calibrate_wasb import make_predict_fn,
load_golden_gts, BASELINE
backend\eval\export_wasb_segments.py:37:TUNED: Config = (0.05, 1.0, 1.0)
backend\eval\export_wasb_segments.py:39:SEG_FIELDS = ["rally_number", "start_time", "end_time",
"duration", "start_mmss", "end_mmss"]
backend\eval\export_wasb_segments.py:40:COMP_FIELDS = ["type", "ref", "start_time", "end_time",
"mmss", "status",
backend\eval\export_wasb_segments.py:44:def seconds_to_mmss(t: float) -> str:
backend\eval\export_wasb_segments.py:78:        w = csv.DictWriter(f, fieldnames=SEG_FIELDS)
backend\eval\export_wasb_segments.py:84:def build_comparison(preds: List[Interval], gts:
List[Interval],
backend\eval\export_wasb_segments.py:128:        w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
backend\eval\export_wasb_segments.py:139:def run(trajectory_csv: str, fps: float, frame_width:
float, cfg: Config = TUNED,
backend\eval\export_wasb_segments.py:164:        base_segs = _gate(predict_fn(BASELINE),
"baseline")
backend\eval\export_wasb_segments.py:167:        print(f"baseline cfg {BASELINE}:
{len(base_segs)} segments -> {base_out}")
backend\eval\export_wasb_segments.py:191:def _maybe_slice(video: str, segments_csv: str,
clips_dir: str) -> None:
backend\eval\export_wasb_segments.py:203:    ap.add_argument("--velocity-thresh", type=float,
default=TUNED[0])
backend\eval\export_wasb_segments.py:204:    ap.add_argument("--merge-gap", type=float,
default=TUNED[1])
backend\eval\export_wasb_segments.py:205:    ap.add_argument("--min-duration", type=float,
default=TUNED[2])
backend\eval\gemini_refine.py:30:from backend.eval.calibrate_wasb import make_predict_fn,
BASELINE
backend\eval\gemini_refine.py:31:from backend.eval.export_wasb_segments import TUNED,
write_segments_csv, seconds_to_mmss
backend\eval\gemini_refine.py:102:        cfg=BASELINE, pad: float = 3.0, model: str = "gemini-
flash-latest",
backend\eval\gemini_refine.py:226:    cfg = BASELINE if args.config == "baseline" else TUNED
backend\eval\audio\e1_probe.py:19:from backend.eval.calibrate_wasb import load_golden_gts,
make_predict_fn, BASELINE
backend\eval\audio\e1_probe.py:57:    wasb = make_predict_fn(trajectory, fps,
frame_width)(BASELINE)
backend\eval\regression.py:30:DEFAULT_BASELINE_PATH = os.path.join("output",
"regression_baseline.json")
backend\eval\regression.py:55:def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str,
dict]:
backend\eval\regression.py:68:                  f1: float, path: str = DEFAULT_BASELINE_PATH) ->
None:
backend\eval\regression.py:91:            baseline_path: str = DEFAULT_BASELINE_PATH) ->
RegressionResult:
backend\eval\regression.py:133:                          baseline_path: str =
DEFAULT_BASELINE_PATH,
```

35. assistant (2026-06-07T09:18:34.856Z)

All eval module symbols and constants verified, including `BASELINE=(0.02,2.0,2.0)`, `TUNED=(0.05,1.0,1.0)`, `BASELINE_LOCAL`, the cross-imports. Now let me verify the remaining

subsystems: annotations, providers, sports, stitcher, tools, utils, calibration, classifier, training, distill_local, gemini_refine, audio, and the entrypoint scripts.

36. user (2026-06-07T09:18:37.153Z)

```
backend\eval\calibration.py:34:def pct_improvement(before: float, after: float) -> float:
backend\eval\calibration.py:46:def kfold_indices(n: int, k: int) -> List[Tuple[List[int],
List[int]]]:
backend\eval\calibration.py:61:def select_best(predict_fn: PredictFn, configs: Sequence[Config],
gts: List[Interval],
backend\eval\calibration.py:72:def cross_validate(predict_fn: PredictFn, configs:
Sequence[Config], gts: List[Interval],
backend\eval\calibration.py:113:def leave_one_video_out(videos: List[Dict[str, Any]], configs:
Sequence[Config],
backend\eval\calibration.py:148:def improvement_report(predict_fn: PredictFn, baseline_config:
Config, tuned_config: Config,
backend\eval\gemini_refine.py:39:def merge_overlaps(intervals: List[Interval], gap_tol: float =
0.0) -> List[Interval]:
backend\eval\gemini_refine.py:71:def refine_window(api_key: str, model: str, sport_profile,
video: str, window: Interval,
backend\eval\gemini_refine.py:149:def full_video_rallies(video: str, model: str = "gemini-flash-
latest",
backend\eval\distill_local.py:41:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity",
"active_ratio", "net_crossings"]
backend\eval\distill_local.py:48:def build_candidates(trajectory_csv: str, fps: float,
frame_width: float,
backend\eval\distill_local.py:90:def predict_rallies(model: LogisticRegression, X:
List[List[float]], intervals: List[Interval],
backend\eval\distill_local.py:116:            model = LogisticRegression().fit(X, y,
FEATURE_NAMES)
backend\eval\distill_local.py:134:    final = LogisticRegression().fit(Xall, yall,
FEATURE_NAMES)
backend\eval\distill_local.py:135:    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for
w in final.w]))
backend\eval\classifier.py:21:class LogisticRegression:
backend\eval\audio\e1_probe.py:26:def cluster_hits(hits: List[HitEvent], gap_sec: float = 2.0,
min_hits: int = 2,
backend\eval\training.py:28:YOLO_FEATURE_NAMES = [
backend\eval\training.py:33:def extract_yolo_features(row: Dict) -> List[float]:
backend\eval\training.py:50:def label_candidates(candidate_intervals: List[Interval],
gt_intervals: List[Interval],
backend\eval\training.py:64:def build_training_set(
```

37. user (2026-06-07T09:18:41.098Z)

```
backend\main.py:170:def compile_highlights(req: CompileRequest):
backend\pipeline\audio\hit_detector.py:23:class HitEvent:
backend\pipeline\audio\hit_detector.py:28:def extract_audio(video_path: str, out_wav: str =
None, sr: int = 16000) -> str:
backend\pipeline\audio\hit_detector.py:40:def load_wav(path: str) -> Tuple[np.ndarray, int]:
backend\pipeline\audio\hit_detector.py:64:def onset_envelope(samples: np.ndarray, sr: int,
frame: int = 1024,
backend\pipeline\audio\hit_detector.py:106:def detect_hits(samples: np.ndarray, sr: int, k:
float = 2.5, win_frames: int = 41,
backend\pipeline\audio\hit_detector.py:136:def detect_hits_in_video(video_path: str, sr: int =
16000, **kw) -> List[HitEvent]:
backend\utils\video.py:8:def get_video_duration(video_path: str) -> float:
backend\utils\video.py:21:def extract_video_chunk(video_path: str, start_time: float, duration:
float,
backend\pipeline\stitcher\compiler.py:6:class VideoCompiler:
backend\pipeline\stitcher\compiler.py:14:    def _parse_time(val: Union[int, float, str]) ->
float:
backend\pipeline\stitcher\compiler.py:43:    def _get_video_duration(self, video_path: str) ->
float:
backend\pipeline\stitcher\compiler.py:61:    def compile_highlights(self, raw_video_path: str,
```

```
segments: List[Tuple[float, float]], output_filename: str) -> str:
backend\utils\geometry.py:23:def get_velocity_normalised(bbox1: Tuple[float, float, float,
float], bbox2: Tuple[float, float, float, float],
backend\utils\geometry.py:42:def calculate_iou(boxA: Tuple[float, float, float, float], boxB:
Tuple[float, float, float, float]) -> float:
backend\tools\rally_slicer.py:30:BEGIN_PADDING = 0.5
backend\tools\rally_slicer.py:31:END_PADDING = 1.0
backend\tools\rally_slicer.py:35:class ClipWindow:
backend\tools\rally_slicer.py:53:def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float,
float]]:
backend\tools\rally_slicer.py:76:def compute_clip_windows(labels: Dict[str, Tuple[float,
float]],
backend\tools\rally_slicer.py:78:                          begin_pad: float = BEGIN_PADDING,
backend\tools\rally_slicer.py:79:                          end_pad: float = END_PADDING,
backend\tools\rally_slicer.py:118:def slice_rallies(video_path: str, labels_csv: str,
rally_numbers: List[str],
backend\tools\rally_slicer.py:119:                  out_dir: str, begin_pad: float =
BEGIN_PADDING,
backend\tools\rally_slicer.py:120:                  end_pad: float = END_PADDING) -> List[str]:
backend\tools\rally_slicer.py:165:    p.add_argument("--begin-pad", type=float,
default=BEGIN_PADDING)
backend\tools\rally_slicer.py:166:    p.add_argument("--end-pad", type=float,
default=END_PADDING)
```

38. assistant (2026-06-07T09:18:46.720Z)

All those symbols verified. Note CODE_MAP §2.4 says compiler `end_padding`(1.0) default ? but let me check: the model StitchingConfig has `end_padding=0.5`. The CODE_MAP §2.4 claims "begin_padding(0.5)/end_padding(1.0) defaults" for the compiler. Let me check the compiler's actual hardcoded defaults, and also verify a couple of CODE_MAP claims that I haven't pinned: the `calibration.evaluate`, `pr_curve`, plus the remaining annotations/providers/sports symbols.

39. user (2026-06-07T09:18:48.750Z)

```
61          def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
62              """
63              Extracts selected segments from a raw video and stitches them together
64              into a seamless highlights file, applying end_padding to prevent abrupt endings.
65              """
66              if not segments:
67                  raise ValueError("No highlight segments provided to compile.")
68
69              if not os.path.exists(raw_video_path):
70                  raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
71
72              output_path = os.path.join(self.output_dir, output_filename)
73
74              # Probe video duration so we can detect out-of-range segments
75              video_duration = self._get_video_duration(raw_video_path)
76              if video_duration > 0:
77                  print(f"[COMPILER] Source video duration: {video_duration:.2f}s")
78              else:
79                  print("[COMPILER WARNING] Video duration unknown. Cannot validate segment
boundaries against video length.")
80
81              # We will create temporary clips and stitch them using FFmpeg concat demuxer
82              temp_clips = []
83              skipped_segments = []
84
85              # Create a temp directory within output_dir
86              with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
87                  print(f"[COMPILER] Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
```

```
88                  for idx, (raw_start, raw_end) in enumerate(segments):
89                      # Normalize timestamps to float seconds ? handles float, int,
90                      # plain numeric strings, and colon-separated formats like "MM:SS"
91                      start = self._parse_time(raw_start)
92                      end = self._parse_time(raw_end)
93                      segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
94
95                      # Validate the segment times
96                      if start < 0:
97                          print(f"[COMPILER WARNING] {segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
98                          start = 0.0
99
100                     if start >= end:
```

40. user (2026-06-07T09:18:50.881Z)

```
backend\annotations\__init__.py:7:from .base import annotation_registry, AnnotationProvider
backend\annotations\__init__.py:11:__all__ = ["annotation_registry", "AnnotationProvider"]
backend\annotations\base.py:28:class AnnotationProvider(ABC):
backend\annotations\base.py:81:class AnnotationProviderRegistry:
backend\annotations\base.py:110:annotation_registry = AnnotationProviderRegistry()
backend\annotations\file_provider.py:16:    annotation_registry,
backend\annotations\file_provider.py:24:class FileProvider(AnnotationProvider):
backend\annotations\file_provider.py:66:annotation_registry.register(FileProvider)
backend\annotations\automated_provider.py:24:    annotation_registry,
backend\annotations\automated_provider.py:45:        "annotation_registry.get('auto',
labeler=<fn>, lineage='<model-family>'). "
backend\annotations\automated_provider.py:50:def warn_if_same_lineage(oracle_lineage:
Optional[str], production_lineage: Optional[str]) -> bool:
backend\annotations\automated_provider.py:66:class AutomatedProvider(AnnotationProvider):
backend\annotations\automated_provider.py:103:annotation_registry.register(AutomatedProvider)
backend\eval\calibration.py:31:PredictFn = Callable[[Config], List[Interval]]
backend\eval\calibration.py:41:def evaluate(preds: List[Interval], gts: List[Interval],
iou_threshold: float = 0.5) -> Score:
backend\eval\calibration.py:61:def select_best(predict_fn: PredictFn, configs: Sequence[Config],
gts: List[Interval],
backend\eval\calibration.py:72:def cross_validate(predict_fn: PredictFn, configs:
Sequence[Config], gts: List[Interval],
backend\eval\calibration.py:117:    `videos`: list of {"name": str, "predict_fn": PredictFn,
"gts": [Interval]}.
backend\eval\calibration.py:148:def improvement_report(predict_fn: PredictFn, baseline_config:
Config, tuned_config: Config,
backend\eval\gemini_refine.py:10:Reuses (no reinvented wheels): providers.GeminiProvider (via
provider_registry),
backend\eval\gemini_refine.py:11:sports.badminton prompt (sport_registry),
utils.video.extract_video_chunk,
backend\eval\gemini_refine.py:33:from backend.sports import sport_registry
backend\eval\gemini_refine.py:34:from backend.providers import provider_registry
backend\eval\gemini_refine.py:76:    provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
backend\eval\gemini_refine.py:108:    sport_profile = sport_registry.get("badminton")
backend\eval\gemini_refine.py:159:    sport_profile = sport_registry.get("badminton")
backend\eval\gemini_refine.py:174:            provider = provider_registry.get("gemini",
api_key=api_key, model_name=model)
backend\sports\badminton.py:3:class BadmintonSportProfile(SportProfile):
backend\eval\harness.py:66:def evaluate(db: Database, video_id: str, gt_intervals:
List[Interval],
backend\sports\base.py:3:class SportProfile(ABC):
backend\sports\__init__.py:5:class SportRegistry:
backend\sports\__init__.py:23:sport_registry = SportRegistry()
backend\eval\regression.py:26:from backend.annotations import annotation_registry
backend\eval\regression.py:144:    provider = annotation_registry.get(tier, **(provider_kwargs
or {}))
```

```
backend\pipeline\segmenters\gemini.py:11:from backend.sports import sport_registry
backend\pipeline\segmenters\gemini.py:12:from backend.providers import provider_registry
backend\pipeline\segmenters\gemini.py:33:            sport_profile =
sport_registry.get(sport_name)
backend\pipeline\segmenters\gemini.py:55:            provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend\providers\base.py:4:class AIVideoProvider(ABC):
backend\pipeline\segmenters\tracknet_hybrid.py:25:from backend.sports import sport_registry
backend\pipeline\segmenters\tracknet_hybrid.py:26:from backend.providers import
provider_registry
backend\pipeline\segmenters\tracknet_hybrid.py:63:            sport_profile =
sport_registry.get(sport_name)
backend\pipeline\segmenters\tracknet_hybrid.py:82:            provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend\providers\gemini.py:14:MAX_UPLOAD_MB = 500
backend\providers\gemini.py:16:class GeminiProvider(AIVideoProvider):
backend\providers\gemini.py:44:        if size_mb > MAX_UPLOAD_MB:
backend\providers\gemini.py:45:            logger.error(f"[{log_prefix}] {video_path} is
{size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap ? "
backend\pipeline\segmenters\wasb_hybrid.py:23:from backend.sports import sport_registry
backend\pipeline\segmenters\wasb_hybrid.py:24:from backend.providers import provider_registry
backend\pipeline\segmenters\wasb_hybrid.py:59:            sport_profile =
sport_registry.get(sport_name)
backend\pipeline\segmenters\wasb_hybrid.py:74:            provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend\providers\__init__.py:5:class ProviderRegistry:
backend\providers\__init__.py:22:provider_registry = ProviderRegistry()
backend\pipeline\segmenters\yolo_hybrid.py:24:from backend.sports import sport_registry
backend\pipeline\segmenters\yolo_hybrid.py:25:from backend.providers import provider_registry
backend\pipeline\segmenters\yolo_hybrid.py:269:            sport_profile =
sport_registry.get(sport_name)
backend\pipeline\segmenters\yolo_hybrid.py:291:            provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend\pipeline\validators\relevance.py:41:        from backend.sports import sport_registry
backend\pipeline\validators\relevance.py:43:            sport_profile =
sport_registry.get(sport_name)
```

41. assistant (2026-06-07T09:18:56.596Z)

All annotations, providers, sports symbols verified. `MAX_UPLOAD_MB=500`, `warn_if_same_lineage`, `FileProvider`/`AutomatedProvider` registered + imported in __init__. The CODE_MAP §2.6 says GeminiProvider has `::MAX_UPLOAD_MB=500` ? it's a module-level const (line 14), but the CODE_MAP places it under the GeminiProvider symbol. Minor placement nuance, not a broken anchor.

Now let me check the compiler's `begin_padding`/`end_padding` defaults and the validators' specific symbols, plus the `__init__` and `class Labeler` for automated_provider.

42. user (2026-06-07T09:18:58.015Z)

```
6       class VideoCompiler:
7           def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
8               self.output_dir = output_dir
9               self.end_padding = end_padding
10              self.begin_padding = begin_padding
11              os.makedirs(output_dir, exist_ok=True)
12
13          @staticmethod
14          def _parse_time(val: Union[int, float, str]) -> float:
15              """Normalizes a timestamp value to float seconds.
16              Handles: float/int (pass-through), plain numeric strings ("12.4"),
17              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
18              This mirrors the parse_time logic used in gemini.py to ensure
```

43. user (2026-06-07T09:19:00.789Z)

```
backend\pipeline\validators\base.py:5:class ValidationResult(BaseModel):
backend\pipeline\validators\base.py:11:class VideoValidator(ABC):
backend\pipeline\validators\base.py:34:class ValidationRegistry:
backend\pipeline\validators\base.py:42:    def run_all_with_context(self, video_path: str,
config: Dict[str, Any]):
backend\pipeline\validators\blur.py:5:class BlurValidator(VideoValidator):
backend\pipeline\validators\blur.py:8:        return "blur_check"
backend\pipeline\validators\format.py:7:class FormatValidator(VideoValidator):
backend\pipeline\validators\format.py:10:        return "format_check"
backend\pipeline\validators\relevance.py:6:class RelevanceValidator(VideoValidator):
backend\pipeline\validators\relevance.py:9:        return "relevance_check"
backend\pipeline\validators\pts_continuity.py:5:class PTSContinuityValidator(VideoValidator):
backend\pipeline\validators\pts_continuity.py:8:        return "pts_continuity_check"
backend\pipeline\validators\scene_cut.py:6:class SceneCutDensityValidator(VideoValidator):
backend\pipeline\validators\scene_cut.py:9:        return "scene_cut_check"
backend\pipeline\validators\resolution_fps.py:4:class ResolutionFPSValidator(VideoValidator):
backend\pipeline\validators\resolution_fps.py:7:        return "resolution_fps_check"
backend\pipeline\validators\resolution_fps.py:19:                message="Video metadata is
missing. Run format_check validator first."
```

44. assistant (2026-06-07T09:19:07.268Z)

The compiler's `__init__` has `end_padding=1.0, begin_padding=0.5` ? confirming CODE_MAP §2.4 "begin_padding(0.5)/end_padding(1.0) defaults". All validators verified with correct names and `*_check` validator_name values. The `run_all_with_context` exists. The resolution_fps validator does hard-fail "Run format_check ... first" (line 19) ? matches.

Two items I noticed: `class Labeler` and `EXPECTED_CANDIDATE_COLUMNS` did not appear in the grep. The CODE_MAP §2.6 references `::Labeler` in automated_provider, and §8 test map references `EXPECTED_CANDIDATE_COLUMNS` (in test_database). Let me verify the `Labeler` symbol in automated_provider (CODE_MAP claims it as a symbol there).

45. user (2026-06-07T09:19:08.786Z)

```
33:Labeler = Callable[[str, Optional[str], str], List[Interval]]
36:def _not_configured(video_ref: str, guideline: Optional[str], sport: str) -> List[Interval]:
66:class AutomatedProvider(AnnotationProvider):
67:    def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
75:        self._labeler = labeler or _not_configured
78:        self._jobs: Dict[str, List[Interval]] = {}
91:        self._jobs[job_id] = intervals
95:        return DONE if job_id in self._jobs else FAILED
98:        if job_id not in self._jobs:
100:         return self._jobs[job_id]
```

46. user (2026-06-07T09:19:09.485Z)

```
58:EXPECTED_CANDIDATE_COLUMNS = {
72:    assert names == EXPECTED_CANDIDATE_COLUMNS
```

47. assistant (2026-06-07T09:19:14.141Z)

Both confirmed: `Labeler` is a type alias in automated_provider, `EXPECTED_CANDIDATE_COLUMNS` is defined in test_database.py (the CODE_MAP §8 references it as the test's guard, correctly). Now let me verify the entrypoint scripts (run_eval, run_phase3_eval, run_regression, run_process_and_stitch), setup.py, run_all_tests.py, and the audio probe / e1 symbols, plus the spec.yaml rules.

48. user (2026-06-07T09:19:15.659Z)

```
31:def get_file_md5(filepath: str) -> str:
40:def _default_db_path() -> str:
```

```
46:                db_name = json.load(f).get("output", {}).get("db_name", db_name)
88:def main(argv=None) -> int:
119:    db_path = args.db or _default_db_path()
```

49. user (2026-06-07T09:19:17.167Z)

```
30:from dotenv import load_dotenv
31:load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
51:def load_config(path: str = "config.json") -> dict:
54:            return json.load(f)
89:def main(argv=None) -> int:
101:        manifest = json.load(f)
```

50. user (2026-06-07T09:19:18.224Z)

```
10:    python run_regression.py --video-id <md5> --tier file --annotations rallies.csv --update-
baseline
26:def _default_db_path() -> str:
31:                db_name = json.load(f).get("output", {}).get("db_name", db_name)
49:    p.add_argument("--update-baseline", action="store_true",
55:def main(argv=None) -> int:
59:    db_path = args.db or _default_db_path()
65:    result = regression.regress_with_provider(
```

51. assistant (2026-06-07T09:19:24.096Z)

All entrypoints verified. run_eval has `_default_db_path`, `get_file_md5`, `main`, raw `json.load`. run_phase3_eval has its own `load_config` raw loader, `load_dotenv()`, `main`. run_regression has `_default_db_path`, `--update-baseline`, `regress_with_provider`.

Let me confirm the `phase3` `segmenter_cls(db, cfg)` with raw dict claim, the phase3 chunk size, and verify run_process_and_stitch + run_all_tests.py + setup.py + spec.yaml rules. Also need to check phase3 uses `default_segmenter` fallback 'motion' per CODE_MAP.

52. user (2026-06-07T09:19:24.984Z)

```
89      def main(argv=None) -> int:
90          logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
91          args = build_parser().parse_args(argv)
92
93          manifest_path = os.path.abspath(args.manifest)
94          if not os.path.exists(manifest_path):
95              print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
96              return 2
97          manifest_dir = os.path.dirname(manifest_path)
98          videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
99
100         with open(manifest_path) as f:
101             manifest = json.load(f)
102         entries = manifest.get("eval_sets", [])
103
104         cfg = load_config()
105         db_name = cfg.get("output", {}).get("db_name", "sports_indexer.db")
106         db_path = args.db or os.path.join("output", db_name)
107         os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
108         db = Database(db_path)
109
110         stages = batch.default_stages_for(args.segmenter, args.stage)
111
112         # Resolve the segmenter class up front (unless purely scoring).
113         segmenter = None
114         if not args.score_only:
115             seg_cls = segmenter_registry.get(args.segmenter)
116             if not seg_cls:
```

```
117                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
118                       f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
119                 return 2
120             segmenter = seg_cls(db, cfg)
121 
122         # Filter entries.
123         targets = []
124         for e in entries:
125             if args.only and e["video_id"] not in args.only:
126                 continue
127             targets.append(e)
128         if args.limit:
129             targets = targets[: args.limit]
130 
131         per_video = []
132         skipped = []
133         print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
134 
135         for e in targets:
136             vid = e["video_id"]
137             video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
138             csv_path = os.path.join(manifest_dir, e["csv"])
139 
140             if not video_path or not os.path.exists(video_path):
141                 skipped.append((vid, "no video file"))
142                 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
143                 continue
144             if not os.path.exists(csv_path):
145                 skipped.append((vid, "no GT csv"))
146                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
147                 continue
148 
149             video_id = get_file_md5(video_path)
150 
151             # Segment unless score-only or already present (and not --reprocess).
152             if not args.score_only:
153                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
154                 if already and not args.reprocess:
155                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
156                 else:
157                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
158                     db.add_video(video_id, video_path, "processed", [])
159                     t0 = time.time()
160                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
161                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
162 
163             gt = load_annotations(csv_path)
164             report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
165             print(format_report_text(report, show_failures=args.show_failures))
166             print()
167             d = report_to_dict(report)
168             d["collector_video_id"] = vid
169             per_video.append(d)
170 
171         agg = batch.aggregate(per_video, stages)
172         print(batch.format_aggregate(agg))
173         if skipped:
174             print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
```

```
175
176         if args.json_out:
177             with open(args.json_out, "w") as f:
178                 json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
```

53. assistant (2026-06-07T09:19:30.580Z)

This confirms phase3 uses `seg_cls(db, cfg)` with a raw dict `cfg` (from its own `load_config` json.load). Note: CODE_MAP §2.0 says phase3 "DB key = file MD5, NOT manifest's `video_id`" ? confirmed (line 149 `video_id = get_file_md5(video_path)` while `vid = e["video_id"]` at line 136).

Let me check the phase3 `--segmenter` default value (CODE_MAP claims `'motion'`), the get_file_md5 chunk size, the spec.yaml rules, and run_process_and_stitch + run_all_tests + setup.

54. user (2026-06-07T09:19:33.042Z)

```
46:         for chunk in iter(lambda: f.read(65536), b""):
64:     p.add_argument("--videos-root", default=None,
67:     p.add_argument("--segmenter", default="motion",
70:     p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
72:     p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
73:     p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
74:     p.add_argument("--only", action="append", default=None,
76:     p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
77:     p.add_argument("--max-frames", type=int, default=None,
83:     p.add_argument("--db", default=None, help="Indexer DB path (default: output/<db_name>).")
84:     p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report here.")
```

55. user (2026-06-07T09:19:34.055Z)

```
28:         for chunk in iter(lambda: f.read(1024 * 1024), b""):
44:             config = json.load(f)
65:     use_cache = stitching_config.get("use_cache", True)
75:             choice = input("Would you like to (1) Run stitching on the cached segments, or
(2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
92:         seg_name = config.get("indexing", {}).get("default_segmenter", "yolo_hybrid")
114:             bpad_input = input(f"Enter begin_padding in seconds (default {begin_padding}):
").strip()
118:             pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
122:             shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
137:                 meta = json.loads(meta)
```

56. user (2026-06-07T09:19:35.832Z)

```
9:customer_verdict:
26:  - code: silent_provider_noop
44:  - code: ai_empty_vs_no_rallies
58:  - code: fps_fallback_used
72:  - code: synthetic_motion_metadata
85:  - code: segmenter_divergence
99:  - code: reingest_replaced_prior
111:  - code: validation_failed
```

57. assistant (2026-06-07T09:19:42.154Z)

All confirmed:
- phase3 `--segmenter` default `"motion"` (line 67) and `get_file_md5` chunk 65536/64KB (line

46) ? matches CODE_MAP.
- run_process_and_stitch: chunk `1024*1024`/1MB (line 46 of main.py says 64KB; CODE_MAP says "1MB run_process" ? confirmed line 28 here), `use_cache` default `True` (line 65), `input()` blocking (line 75), `default_segmenter` fallback `'yolo_hybrid'` (line 92), raw `json.load` (line 44) ? all match.
- spec.yaml has all 7 rules + `customer_verdict` ? confirmed.

Now let me verify run_all_tests.py exists at repo root, setup.py symbols, the audio e1_probe `::run`/`::main`, hit_detector `_moving_stats`, and the `__init__.py` for annotations (noqa F401 imports).

58. user (2026-06-07T09:19:42.749Z)

run_all_tests.py

59. user (2026-06-07T09:19:44.180Z)

```
11:def run_pip_install():
16:    has_gpu = check_command("nvidia-smi")
35:def init_default_config():
42:        from backend.config import save_default_config
54:        saved_path = save_default_config(config_path)
59:def run_diagnostics():
```

60. user (2026-06-07T09:19:45.093Z)

```
26:def cluster_hits(hits: List[HitEvent], gap_sec: float = 2.0, min_hits: int = 2,
53:def run(video: str, labels_csv: str, trajectory: str, fps: float, frame_width: float,
98:def main() -> None:
```

61. user (2026-06-07T09:19:45.554Z)

```
1       """Pluggable sources of ground-truth rally annotations.
2
3       `AnnotationProvider`s yield `(start, end)` intervals ? the same shape the eval
4       harness scores ? so golden-set labels can come from a file, a human vendor, or an
5       automated oracle behind one interface. See `docs/GOLDEN_SET_VENDORING.md`.
6       """
7       from .base import annotation_registry, AnnotationProvider
8       from . import file_provider       # noqa: F401  (side-effect: registers FileProvider)
9       from . import automated_provider  # noqa: F401  (side-effect: registers
AutomatedProvider)
10
11      __all__ = ["annotation_registry", "AnnotationProvider"]
12
```

62. assistant (2026-06-07T09:19:51.885Z)

All confirmed:
- `run_all_tests.py` exists at repo root.
- setup.py: `init_default_config` delegates to `save_default_config` (line 42, 54), `run_pip_install` (line 11) GPU-aware via nvidia-smi (line 16), `run_diagnostics` (line 59) ? matches.
- e1_probe: `cluster_hits`, `run`, `main` ? confirmed.
- annotations __init__ noqa F401 side-effect imports ? confirmed.

Now let me verify the test map references are accurate. The CODE_MAP claims a stale `.pyc` only `tests/__pycache__/test_gemini_api.*` with no live source. Let me verify there's no `test_gemini_api.py` source, and check the audio hit_detector `_moving_stats` symbol, plus the `pr_curve` / `Match` in metrics, and the `gemini.py` segmenter symbols (parse_time, GeminiPlaySegmenter).

63. user (2026-06-07T09:19:52.460Z)

```
No files found
```

64. user (2026-06-07T09:19:53.929Z)

```
test_gemini_api.cpython-311-pytest-9.0.3.pyc
test_gemini_api.cpython-313-pytest-9.0.3.pyc
```

65. user (2026-06-07T09:19:55.278Z)

```
64:def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
91:def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
115:    mean, std = _moving_stats(env, win_frames)
```

66. user (2026-06-07T09:19:56.386Z)

```
16:class GeminiPlaySegmenter(BasePlaySegmenter):
193:            with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
269:        def parse_time(val) -> float:
302:        max_segment_duration = 90.0  # Singles play segments almost never exceed this
344:            if dur > max_segment_duration:
346:                    f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
```

67. assistant (2026-06-07T09:20:03.027Z)

```
All confirmed:
- No `test_gemini_api.py` source exists, but stale `.pyc` files DO exist in `tests/__pycache__/`
? CODE_MAP §8 "stale `.pyc` only `tests/__pycache__/test_gemini_api.*` ? no live source exists"
is accurate.
- `_moving_stats` and `onset_envelope` in hit_detector ? confirmed.
- gemini.py: `GeminiPlaySegmenter`, `parse_time`, `max_workers=5` hardcoded (line 193),
`max_segment_duration=90.0` hardcoded (line 302) ? all match.

The CODE_MAP is comprehensive and accurate. Let me do a final spot-check on a few remaining
CODE_MAP claims I want to confirm: the `rally_gate` consumers
(calibrate_local/distill_local/export_wasb_segments import it), the static UI path
`backend/api/static`, and verify the `tests/` files all exist as claimed in the test map by
cross-referencing the git ls-files list (already done ? all present). Let me check the
rally_gate import claim and the metrics `Match`/`pr_curve` symbols.
```

68. user (2026-06-07T09:20:04.075Z)

```
backend\eval\calibrate_local.py:7:                        -> rally_gate (min_crossings)
-> rally windows
backend\eval\calibrate_local.py:15:Reuses: TrackNetRunner windowing, rally_gate,
calibrate_wasb.load_golden_gts,
backend\eval\calibrate_local.py:30:from backend.pipeline.detectors import rally_gate
backend\eval\calibrate_local.py:52:            wins = rally_gate.apply_gate(points, wins, fps,
min_crossings=mc)
backend\eval\export_wasb_segments.py:30:from backend.pipeline.detectors import rally_gate
backend\eval\export_wasb_segments.py:151:        kept = rally_gate.apply_gate(points, segs, fps,
min_crossings=min_crossings)
backend\eval\distill_local.py:18:rally_gate, metrics, calibrate_wasb.load_golden_gts,
gemini_refine.merge_overlaps.
backend\eval\distill_local.py:32:from backend.pipeline.detectors import rally_gate
backend\eval\distill_local.py:57:    gated = rally_gate.gate_windows(points, intervals, fps,
min_crossings=0)  # for net-crossings
```

69. user (2026-06-07T09:20:04.347Z)

```
backend\api\static\app.js
backend\api\static\index.html
backend\api\static\style.css
```

70. assistant (2026-06-07T09:20:17.150Z)

All confirmed:
- `rally_gate` consumed by calibrate_local, export_wasb_segments, distill_local (CODE_MAP §2.2 says "Consumed by calibrate_local / distill_local / export_wasb_segments") ? confirmed.
- `backend/api/static` directory exists with app.js/index.html/style.css ? confirms the CODE_MAP §2.0 static UI mount claim.

I have thoroughly verified the CODE_MAP. Let me summarize the verification:

Directory structure / path migrations: The repo uses the NEW layout (`backend/pipeline/{segmenters,detectors,validators,stitcher,audio}/`, `backend/infrastructure/database.py`, `backend/config/`, `backend/reporting/`). The CODE_MAP consistently references these new paths. I searched the entire CODE_MAP text for OLD paths (`backend/validators`, `backend/indexer`, `backend/database`, `backend/stitcher`, `backend/detectors`, `backend/audio`) ? **none appear** as anchors; every `@path` uses the correct `pipeline/`/`infrastructure/` prefix.

All @path anchors: Every file referenced exists and is git-tracked (cross-checked against `git ls-files`).

All ::symbols: Verified across config, main, segmenters, detectors (tracknet/wasb runners, wasb_infer, rally_gate), eval (metrics, harness, regression, calibration, calibrate_wasb/local, export, batch, training, classifier, distill_local, gemini_refine, audio/e1_probe), stitcher, tools, utils, validators, sports, providers, annotations, infrastructure, reporting (init, harvest, spec.yaml). All present.

Key gotcha claims spot-checked and accurate: 64KB/1MB/8KB chunk drift, `output_dir` runtime-only with no OutputConfig field, segmenter default in 5 places/3 values, `AppConfig.get` returns dict sections, `INSERT OR REPLACE` cascade, validator exec order with format?resolution_fps dependency, wasb/tracknet shot_count drift, staticmethod rebinding, `_PERSON_CLASS_ID=1`, stale `.pyc` for test_gemini_api with no source.

Nothing is broken. Returning the result.

71. user (2026-06-07T09:20:17.235Z)

Structured output provided successfully

72. assistant (2026-06-07T09:20:30.622Z)

Verification complete. The regenerated CODE_MAP is accurate against the real repo ? `ok=true`, no problems found.

Summary of what I checked:

- **All `@path` anchors resolve and are git-tracked.** Cross-referenced every file path in the CODE_MAP against `git ls-files`. The repo uses the new layout (`backend/pipeline/{segmenters,detectors,validators,stitcher,audio}/`, `backend/infrastructure/database.py`, `backend/config/`, `backend/reporting/`), and the CODE_MAP references these correctly throughout.

- **No OLD-path references.** Scanned the entire CODE_MAP for `backend/validators`, `backend/indexer`, `backend/database`, `backend/stitcher`, `backend/detectors`, `backend/audio` used as anchors ? none appear. Every anchor uses the correct `pipeline/` / `infrastructure/` prefix.

- **All `::symbols` exist** in their stated files: config (loader/models/__init__), main, the five segmenters (names `wasb_hybrid`/`tracknet_hybrid`/`yolo_hybrid`/`gemini`/`motion` all verified), detectors (tracknet_runner staticmethods, wasb_runner staticmethod rebind, wasb_infer cache symbols, rally_gate), the full eval stack, stitcher/tools/utils, validators (all `*_check` names), sports/providers/annotations registries, infrastructure DB, and reporting (init/harvest/spec.yaml's 7 rules + verdict).

- **Load-bearing gotcha claims spot-checked as true**: MD5 chunk-size drift (64KB

main/phase3/run_eval, 1MB run_process), `output_dir` runtime-only (OutputConfig has no such field; `compile_highlights` falls back to `"output"`), segmenter-default divergence (model `yolo_hybrid` / config.json `gemini` / main fallback `motion` / phase3 default `motion` / process_and_stitch `yolo_hybrid`), `AppConfig.get` returning dict sections, DB `INSERT OR REPLACE` + per-connection FK pragma, validator exec order with the format?resolution_fps dependency, wasb-vs-tracknet `shot_count` `is None` drift, `_PERSON_CLASS_ID=1`, and the stale `test_gemini_api.*.pyc` with no live source (both confirmed present in `tests/__pycache__/`, no `.py`).